

Python Professional — All Days Combined (Day 1–7)

本文档将 7 天 Python 高级训练营的全部内容合并为一个文件，并附有完整的目录，方便导航查阅。

目录

Day 1 — 语言基础速通

- 一、Python 的“哲学”——先理解为什么
- 二、核心数据类型
- 三、字符串 (str)
- 四、容器类型
- 五、控制流
- 六、函数
- 七、异常处理
- 八、面向对象 (OOP) 基础
- 九、文件 I/O 与 pathlib
- 十、模块与 import

Day 2 — 核心编程范式

- 一、迭代器 (Iterator)
- 二、闭包 (Closure)
- 三、装饰器 (Decorator)
- 四、生成器 (Generator) 和 yield
- 五、上下文管理器 (Context Manager)
- 六、五个概念之间的关系

Day 3 — 标准库深度实战

- 一、collections — 比内置容器更强大的工具
- 二、itertools — 迭代器瑞士军刀
- 三、functools — 函数工具箱
- 四、错误处理模式
- 五、logging — 专业日志
- 六、正则表达式速查

Day 4 — 高级 OOP 与类型系统

- 一、属性控制与内存优化
- 二、接口与类型系统
- 三、数据类与类型注解

Day 5 — 并发编程

- 一、理解并发：为什么 Python 的并发不一样
- 二、threading — 多线程
- 三、multiprocessing — 多进程

- 四、asyncio — 异步 I/O

Day 6 — 工程化工具链

- 一、项目结构 — 从一开始就做对
- 二、pytest — 专业测试框架
- 三、CLI 工具 — Click / Typer
- 四、FastAPI 入门
- 五、SQLAlchemy — 数据库 ORM

Day 7 — 实战项目：URL Shortener

- 一、项目规划
- 二、数据库层 — SQLAlchemy
- 三、数据校验层 — Pydantic
- 四、工具函数层
- 五、应用入口 — FastAPI
- 六、CLI 管理工具 — Click
- 七、pyproject.toml + main.py
- 八、测试
- 九、运行
- 十、扩展方向

Day 1 — 语言基础速通（完整版）

你有多多年编程经验且接触过 Python，不讲“变量是什么”。直接按 Python 和其他语言差异最大的点来，快速建立完整的 Python 心智模型。

一、Python 的“哲学”——先理解为什么

1. 是什么

打开 Python 终端执行 `import this`，会看到 Tim Peters 写的《Python 之禅》——这是 Python 语言设计的核心指导思想。

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Readability counts.
```

2. 解决了什么问题

Python 的哲学解决了一个元问题：当语言设计遇到选择时，该往哪个方向走？

对比一下：

```
# 不 Pythonic (像 Java 翻译过来)  
def calculate_value(a, b, c):
```

```
list_object = new_list()
list_object.append(a)
return_value = 0
for i in list_object:
    return_value = return_value + i
return return_value
```

```
# Pythonic
def calculate(a, b, c):
    return sum([a, b, c])
```

Python 的哲学选择”显式优于隐式”、“可读性优先”。这意味着：- API 设计通常用清晰的名称，而不是缩写 - 没有花括号——缩进本身就是代码块 - 不搞隐式类型转换——宁可报错也不猜

3. 核心理论

Python 的三个核心设计决策贯穿了所有语法：

决策 1：动态强类型

```
# 动态：变量不需要声明类型
x = 42
x = "hello" # 合法

# 强：不会隐式类型转换
"answer: " + 42 # ✗ TypeError
"answer: " + str(42) # ✓
```

跟 JS（弱类型）对比：

```
"5" + 3 = "53" // JS 自动转
"5" - 3 = 2 // JS 自动转
```

Python 不会做这种事。

决策 2：缩进即块结构

```
for i in range(3):
    if i % 2 == 0:
        print(f"{i} is even")
    else:
        print(f"{i} is odd")
print("done") # 在 for 外面
```

标准约定：4 个空格，永远不要混用 Tab。

决策 3：一切皆对象

```
def hello():
    return "hello"

hello.custom_attr = 42 # 函数可以动态加属性
print(hello.__name__) # "hello"

# 函数可以传给变量、作为返回值
def call_twice(func):
    return func() + func()

print(call_twice(hello)) # "hellohello"
```

对”一切皆对象”的深入理解，是打通 Python 高级主题（装饰器、元类、描述器）的关键。

二、核心数据类型

1. 是什么

Python 的内置数据类型比大多数语言丰富，而且每个都有独特的设计。

类型	可否变	可哈希	一句话
int	—	✓	任意精度，不会溢出
float	—	✓	底层 C double，有精度问题
complex	—	✓	原生复数
bool	—	✓	int 子类，True=1
str	✗	✓	Unicode，不可变
list	✓	✗	最常用容器
tuple	✗	✓	不可变列表
set	✓	✗	无序不重复
dict	✓	✗	关联数组
NoneType	—	—	有且只有 None

2. 解决了什么问题

每个类型解决了一个具体的痛点：

- **int 任意精度** — 不用再考虑整数溢出（其他语言的老大难）
- **bool 是 int 子类** — 可以直接做数学运算，隐式布尔转换让条件判断优雅
- **tuple 可哈希** — 可以作为字典键（list 不行）
- **set O(1) 查找** — 比 list 快得多

3. 核心理论

int — 任意精度整数

```
# 不会溢出
a = 2 ** 1000 # 300+ 位的大整数
print(a)

# 进制直接量
print(0b1010) # 10 (二进制)
print(0o12)   # 10 (八进制)
print(0xA)    # 10 (十六进制)
print(bin(10)) # "0b1010"
print(hex(10)) # "0xa"

# 大整数运算
factorial_100 = 1
for i in range(1, 101):
    factorial_100 *= i
print(factorial_100) # 158 位数字——其他语言早就溢出了
```

float — 底层是 C double

```
# 浮点精度问题（跟所有语言一样，因为 IEEE 754）
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False ✗

# 正确比较方式
print(abs(0.1 + 0.2 - 0.3) < 1e-9) # True
print(round(0.1 + 0.2, 2) == 0.3)   # True

# Decimal 精确小数（用于金融场景）
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3

# 特殊值
float("inf")    # 正无穷
float("-inf")   # 负无穷
float("nan")    # Not a Number
```

complex — 复数（很少语言原生支持）

```
c = 3 + 4j
print(c.real)      # 3.0
print(c.imag)      # 4.0
print(abs(c))      # 5.0（模  $\sqrt{3^2+4^2}$ ）
print(c.conjugate()) # (3-4j)
```

bool — int 的子类

```
# True = 1, False = 0
print(True + True)   # 2
print(False * 10)    # 0

# 关键：隐式布尔转换
# 以下值在条件判断中当作 False：
# None, False, 0, 0.0, "", [], (), {}, set(), range(0)
# 其他值都是 True

# 所以不用写：
if len(items) > 0:
# 直接写：
if items:          # ✓ 更 Pythonic
    process(items)

# 也不要写：
if len(items) == 0:
    print("empty")
# 直接写：
if not items:
    print("empty")
```

None — 空值（类似 null/nil）

```
x = None
print(x is None)    # True ✓ 用 is 比较，不用 ==
print(x == None)    # 也可以但不推荐

# None 不是一个特殊值——它是一个单例对象
# 所有 None 引用指向同一个对象
y = None
print(x is y)       # True
```

4. 场景

场景 1：隐式布尔转换在 API 设计中的影响

```
# 一个函数返回列表——调用方如何判断"没有结果"?
def search_users(query):
    # 如果没找到，返回 [] 还是 None?
    ...
    return results

# 如果返回 []:
users = search_users("nonexistent")
if users:                # [] → False
    process(users)

# 如果返回 None:
users = search_users("nonexistent")
if users is not None and users: # 需要两层判断
    process(users)

# ✓ 最佳实践：函数返回一致的类型结构，让隐式布尔转换帮你
def search_users(query):
    if not query:
        return [] # 返回空列表，而不是 None
    return [user for user in all_users if query in user.name]
```

场景 2：float 精度问题在比较中的陷阱

```
# 写单元测试时容易踩的坑
def test_addition():
    assert 0.1 + 0.2 == 0.3 # ✗ 失败

# ✓ 用 pytest.approx 或 math.isclose
from math import isclose
assert isclose(0.1 + 0.2, 0.3) # ✓

# 直接用 round
assert round(0.1 + 0.2, 10) == round(0.3, 10) # ✓
```

5. 替代方案对比

类型	Python	其他语言	差异
int	任意精度	通常 32/64 位	Python 不会溢出
float	double	double	都一样有问题
bool	int 子类	独立类型	True+True=2
null	None	null/nil/undefined	单例，用 is 比较
decimal	Decimal	BigDecimal	精度可配置

6. 常见坑

```
# 坑 1: is 和 == 混用
a = 256
b = 256
print(a is b) # True - 小整数缓存 (-5~256)

c = 257
d = 257
print(c is d) # False! 超出缓存范围

# ✓ 判断值用 ==，判断是不是同一个对象用 is
```

```
# None 判断一定要用 is

# 坑 2: 浮点的 NaN 不那么 NaN
nan = float("nan")
print(nan == nan)  # False! IEEE 754 规定 NaN ≠ NaN
print(nan is nan)  # True (同一个对象)

# 安全判断
import math
math.isnan(nan)  # True ✓

# 坑 3: 空列表不等于 False
print([] == False)  # False
print(bool([]))     # False
if not []:
    print("empty")  # ✓ 正确

# 坑 4: 十进制字符串 vs 数字
# Decimal("0.1") 和 Decimal(0.1) 不一样!
print(Decimal("0.1"))  # 0.1
print(Decimal(0.1))    # 0.10000000000000000055511151231257827021181583404541015625
```

7. 代码验证

```
# 验证 int 的任意精度
# 计算 2^1000000 最后 10 位
result = pow(2, 1000000)
print(str(result)[-10:])  # 瞬间完成，不会溢出

# 验证 None 是单例
print(None is None)  # True
print(id(None))      # 每次运行都一样
```

三、字符串 (str)

1. 是什么

Python 的字符串是 Unicode 字符的不可变序列。它可以是单引号、双引号、三重引号包裹，功能上完全等价。

2. 解决了什么问题

- 程序要处理文本，而文本编码和多语言是最容易出错的地方
- Python 3 的 str 原生存 Unicode，不再有 Python 2 的“str vs unicode”困惑
- f-string 解决了字符串格式化一直以来的痛点：% 格式化可读性差，.format() 啰嗦

3. 核心理论

f-string — Python 3.6 最实用的特性

```
name = "Leo"
age = 30
pi = 3.1415926

# 基础插值
print(f"Name: {name}, Age: {age}")
```

```

# 任意表达式
print(f"Next year: {age + 1}")
print(f"Pi: {pi:.2f}")          # Pi: 3.14
print(f"Pi: {pi:010.3f}")       # Pi: 000003.142

# 对齐
print(f"|{'left':<10}|")         # |left      |
print(f"|{'right':>10}|")        # |      right|
print(f"|{'center':^10}|")       # |  center  |

# 百分比
print(f"Rate: {0.856:.1%}")     # Rate: 85.6%

# 逗号分隔
print(f"{1234567:,}")           # 1,234,567

# 日期格式化
from datetime import datetime
now = datetime.now()
print(f"{now:%Y-%m-%d %H:%M}")  # 2026-05-23 11:00

# 3.12+ 多行 f-string
data = {"name": "Leo", "score": 95}
print(
    f"Name: {data['name']}\n"
    f"Score: {data['score']}"
)

```

字符串不可变

```

s = "hello"
# s[0] = "H"  # ✗ TypeError
s = "H" + s[1:]  # ✔ 只能重新创建

# 每次"修改"都创建新字符串
s = "a"
s = s + "b"  # 创建 "ab" ("a" 还在, 但可能被 GC)

```

join 而不是 +=

```

words = ["hello", "world", "python"]

# ✗ O(n²) — 每次 + 创建新字符串
result = ""
for s in words:
    result += s + ", "

# ✔ O(n) — 一次性拼接
result = ", ".join(words)  # "hello, world, python"

# join 传入可迭代对象, 比生成器还快
result = "".join(f(x) for x in large_data)

```

4. 场景

场景 1: 大量字符串拼接的性能对比

```

import time

n = 100000
words = ["word"] * n

```



```
# +=
t0 = time.perf_counter()
result = ""
for w in words:
    result += w
print(f"+=: {time.perf_counter() - t0:.3f}s")

# join
t0 = time.perf_counter()
result = "".join(words)
print(f"join: {time.perf_counter() - t0:.3f}s")

# n = 100,000 时, join 快几十倍
```

场景 2: 路径/URL 拼接

```
# 不要用字符串 +
base_url = "https://api.example.com"
endpoint = "/v2/users"
url = base_url + endpoint  # 要小心斜杠

# ✓ 用 pathlib 或 urllib.parse
from urllib.parse import urljoin
url = urljoin(base_url, endpoint)

# 文件路径也一样
from pathlib import Path
data_dir = Path("data")
log_file = data_dir / "logs" / "app.log"  # ✓ 更清晰的路径拼接
```

5. 替代方案对比

方法	示例	什么时候用
+	"hello " + name	少量拼接
join	", ".join(items)	列表/可迭代对象批量拼接
f-string	f"Name: {name}"	绝大多数场景
.format()	"Name: {}".format(name)	模板需要变量和模板分离时
%	"Name: %s" % name	兼容旧代码或 logging
Template	Template("\$name").substitute(...)	用户提供的模板

建议: 日常 90% 用 f-string, 批量用 join, 模板用 .format() 或 Template。

6. 常见坑

```
# 坑 1: 字符串切片不报 IndexError
s = "hi"
print(s[0])  # "h"
print(s[5:10])  # "" — 不报错!
print(s[5])  # ✗ IndexError — 但单个索引会报

# 坑 2: str 转 float 可能比较器失败
"1.5" > "10.0"  # True! 字符串比较是字典序, 不是数值
"1.5" > "1.49"  # True — 字典序 5 > 4
# ✓ 比较前先转成数值
float("1.5") > float("10.0")  # False

# 坑 3: encode/decode 要指定编码
```

```

"中文".encode()           # UTF-8 (默认)
"中文".encode("gbk")       # 如果预期是 GBK
b"\xd6\xd0\xce\xc4".decode("gbk") # "中文"
b"\xe4\xb8\xad\xe6\x96\x87".decode("utf-8") # "中文"

# 坑 4: 单引号和双引号没有区别
'hello' == "hello" # True
# 只是为了嵌套方便: f"He said: {'hello'}"

# 坑 5: 3.12 之前的 f-string 不能嵌套相同引号
# ✗ f"Name: {"Leo"}"
# ✓ f'Name: {"Leo"}'

```

7. 代码验证

```

# 验证 join 的性能优势
import time

words = ["word"] * 50000

def test_plus():
    r = ""
    for w in words:
        r += w
    return r

def test_join():
    return "".join(words)

t1 = time.perf_counter()
test_plus()
print(f"+=: {time.perf_counter()-t1:.3f}s")

t1 = time.perf_counter()
test_join()
print(f"join: {time.perf_counter()-t1:.3f}s")

```

四、容器类型

1. 是什么

Python 有四种内置容器：list、tuple、set、dict。它们覆盖了“有序/无序”×“可变/不可变”的所有组合。

	有序	无序
可变	list	set, dict(key)
不可变	tuple	frozenset

2. 解决了什么问题

- 每种容器针对不同的使用模式做了优化
- 语法简洁——字面量语法 [], (), {}, {k:v} 比其他语言更干净
- 切片、推导式、解包等操作让数据转换堪称艺术

3. 核心理论

列表 (list) — 最常用的容器

```

# 创建
nums = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, [1, 2]] # 可以放不同类型

# 索引和切片
nums[0]      # 1
nums[-1]     # 5 (倒数第一个)
nums[1:3]    # [2, 3] (左闭右开)
nums[:2]     # [1, 3, 5] (步进)
nums[::-1]   # [5, 4, 3, 2, 1] (反转)

# 修改
nums.append(6)      # [1, 2, 3, 4, 5, 6]
nums.extend([7, 8]) # [1, 2, 3, 4, 5, 6, 7, 8]
nums.insert(0, 0)   # [0, 1, 2, 3, 4, 5, 6, 7, 8]
nums.pop()          # 删除并返回最后一个
nums.pop(0)         # 删除并返回索引 0
nums.remove(3)       # 删除第一个值为 3 的元素

```

列表推导式 — Python 标志性语法：

```

# 基本
squares = [x**2 for x in range(10)]
# 等价于：
squares = []
for x in range(10):
    squares.append(x**2)

# 带条件过滤
evens = [x for x in range(20) if x % 2 == 0]

# 嵌套推导 (按 for 的顺序阅读)
matrix = [[i*3 + j for j in range(3)] for i in range(3)]
# [[0, 1, 2], [3, 4, 5], [6, 7, 8]]

# 展开嵌套列表 (从里到外读)
flat = [x for row in matrix for x in row]
# [0, 1, 2, 3, 4, 5, 6, 7, 8]

# 字典列表推导
users = [{"name": n, "score": s} for n, s in zip(names, scores)]

```

元组 (tuple) — 不可变列表

```

# 创建
point = (3, 4)
single = (1,) # 注意逗号! (1) 是 int

# 解包
x, y = point

# 变量交换 (底层就是元组打包/解包)
a, b = b, a

# 星号解包
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2, 3, 4], last=5

# 元组可哈希 → 能用做字典键
d = {(1, 2): "point_a"}

# 命名元组更易读
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])

```

```
p = Point(3, 4)
print(p.x, p.y) # 3 4
```

集合 (set) — 无序不重复

```
s = {1, 2, 3, 3, 2}
print(s)          # {1, 2, 3} (自动去重)

s.add(4)
s.discard(99)     # 删除, 不存在也不报错
s.remove(99)      # ✕ 不存在会报 KeyError

# 集合运算
a = {1, 2, 3}
b = {2, 3, 4}

a & b             # {2, 3}          交集
a | b             # {1, 2, 3, 4}    并集
a - b             # {1}             差集
a ^ b             # {1, 4}          对称差集

# O(1) 查找 vs O(n) 查找
if item in large_set: # O(1) 无论集合多大
if item in large_list: # O(n) 越慢越慢
```

字典 (dict) — 关联数组

```
user = {
    "name": "Leo",
    "age": 30,
    "skills": ["Python", "Go"],
}

# 安全取值
user.get("name")          # "Leo"
user.get("salary", 0)     # 0 (key 不存在返回默认值)
user["salary"]            # ✕ KeyError

# setdefault — key 不存在才设值
user.setdefault("level", "junior")
user.setdefault("level", "senior") # 不会覆盖

# 更新/合并
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
d1.update(d2)             # d1 = {"a":1, "b":3, "c":4}
d3 = d1 | d2              # 3.9+ 合并运算符

# 遍历
for key in user:
    print(key, user[key])

for key, value in user.items():
    print(f"{key}={value}")

# 字典推导
squares = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}
```

4. 场景

场景 1: 去重并保留顺序

```
# list → set → list 会丢失顺序
items = [3, 1, 2, 3, 1, 4]
unique = list(set(items)) # [1, 2, 3, 4] - 顺序变了

# ✓ 去重并保留顺序 (利用字典有序了)
unique = list(dict.fromkeys(items)) # [3, 1, 2, 4]

# 或者用循环
seen = set()
unique = []
for x in items:
    if x not in seen:
        seen.add(x)
        unique.append(x)
```

场景 2: zip 配合 dict 构建映射

```
keys = ["name", "age", "city"]
values = ["Leo", 30, "Adelaide"]

# 两个列表变字典
user = dict(zip(keys, values))
# {'name': 'Leo', 'age': 30, 'city': 'Adelaide'}

# 转置字典
original = {"a": 1, "b": 2, "c": 1}
inverted = {}
for k, v in original.items():
    inverted.setdefault(v, []).append(k)
# {1: ['a', 'c'], 2: ['b']}
```

场景 3: 嵌套列表展开

```
# 任意深度展开
def flatten(nested):
    for item in nested:
        if isinstance(item, (list, tuple)):
            yield from flatten(item)
        else:
            yield item

nested = [1, [2, [3, 4], 5], 6]
list(flatten(nested)) # [1, 2, 3, 4, 5, 6]
```

5. 替代方案对比

场景	方法	效率	可读性
列表 loop 构建	列表推导	✓ 快	✓ 一行
列表 loop 构建	for + append	差不多	啰嗦
字典条件构建	字典推导	✓ 快	✓ 一行
去重	set()	O(n)	✓ 但有损耗
去重且保序	dict.fromkeys	O(n)	推荐
查找存在	in(集合)	O(1)	✓
查找存在	in(列表)	O(n)	✗ 大数据慢

6. 常见坑

```

# 坑 1: 列表的 * 是浅拷贝
matrix = [[0]*3] * 3 # [[0,0,0],[0,0,0],[0,0,0]]
matrix[0][0] = 1
print(matrix) # [[1,0,0],[1,0,0],[1,0,0]] - 三个子列表是同一个!

# ✔ 正确方式
matrix = [[0]*3 for _ in range(3)]

# 坑 2: 元组的逗号 vs 括号
t = (1) # int, 不是元组
t = (1,) # 元组
t = 1, # 元组 (括号可选)

# 坑 3: 集合的空字面量
s = {} # 空字典!
s = set() # 空集合

# 坑 4: dict 的 update 会覆盖
d = {"a": 1, "b": 2}
d.update({"b": 3, "c": 4})
print(d) # {'a': 1, 'b': 3, 'c': 4} - b 被覆盖了

# 坑 5: 列表推导的变量泄漏 (Python 2 问题, 3 已修复)
[x for x in range(10)]
# print(x) # Python 2 会输出 9, Python 3 报 NameError ✔

```

7. 代码验证

```

# 验证 set 的 O(1) 查找
import time

n = 10000000
items_list = list(range(n))
items_set = set(items_list)

target = n - 1

t0 = time.perf_counter()
print(target in items_list)
print(f"list: {time.perf_counter()-t0:.6f}s")

t0 = time.perf_counter()
print(target in items_set)
print(f"set: {time.perf_counter()-t0:.6f}s")

# 百万级数据, list 是微秒级, set 是纳秒级

```

五、控制流

1. 是什么

Python 的控制流除了常见的 if/for/while, 还有三个其他语言没有或很晚才有的特性: for-else、海象操作符 :=、match-case。

2. 解决了什么问题

- **for-else**: 解决“是否找到了/是否完成了”这种需要 flag 变量的常见模式
- **海象操作符**: 消除“先赋值再用值”的重复代码
- **match-case**: 替代冗长的 if-elif 链, 且能匹配数据结构

3. 核心理论

for-else / while-else

else 在循环没有被执行 **break** 时触发：

```
# 找素数
def is_prime(n):
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            print(f"{n} is divisible by {i}")
            break
    else:
        print(f"{n} is prime")
        return True
    return False

# ✓ else 使代码更"扁平"—不需要 maintain 一个 flag 变量

# 实战：找可用端口
for port in [8080, 8081, 8082]:
    if check_port(port):
        print(f"Using port {port}")
        break
else:
    raise RuntimeError("No available port")
```

没有 for-else 的话，需要用 flag：

```
# 没有 for-else
found = False
for port in [8080, 8081, 8082]:
    if check_port(port):
        print(f"Using port {port}")
        found = True
        break
if not found:
    raise RuntimeError("No available port")
```

海象操作符 := (3.8+)

赋值语句和条件判断合二为一：

```
# 场景 1: if 中使用
# 之前
data = get_data()
if len(data) > 10:
    print(f"Got {len(data)} items")

# 之后
if (n := len(data)) > 10:
    print(f"Got {n} items") # n 在 if 块里也能用

# 场景 2: while 循环
# 之前
line = file.readline()
while line:
    process(line)
    line = file.readline()

# 之后
while line := file.readline():
```

```
process(line)
```

```
# 场景 3: 列表推导式过滤
```

```
# 筛选变换后非 None 的结果
```

```
results = [y for x in data if (y := transform(x)) is not None]
```

```
# 场景 4: 正则匹配
```

```
if (match := re.search(pattern, text)):
```

```
    print(f"Found: {match.group()}")
```

限制和规范: - 必须用括号括起来才能用在 if 条件中 if (n := len(x)) > 0: - 不要滥用——只用在消除重复判断的场景

match-case (3.10+)

Python 的 match-case 不是简单的 switch——它可以**模式匹配**（匹配结构而不是值）：

```
# 基础——替代 if-elif
```

```
def http_status(code):
```

```
    match code:
```

```
        case 200:
```

```
            return "OK"
```

```
        case 301 | 302: # 多个值用 |
```

```
            return "Redirect"
```

```
        case 404:
```

```
            return "Not Found"
```

```
        case _: # 默认
```

```
            return "Unknown"
```

```
# 进阶——匹配结构
```

```
def process_command(cmd):
```

```
    match cmd:
```

```
        case ("quit",):
```

```
            return "bye"
```

```
        case ("move", x, y) if 0 <= x <= 100 and 0 <= y <= 100:
```

```
            return f"Moving to ({x}, {y})"
```

```
        case {"type": "user", "name": name, "age": age}:
```

```
            return f"User {name}, {age}"
```

```
        case [int() as x, int() as y]:
```

```
            return f"Coordinates: {x}, {y}"
```

```
        case _:
```

```
            return "Unknown command"
```

```
# 匹配枚举
```

```
from enum import Enum, auto
```

```
class Color(Enum):
```

```
    RED = auto()
```

```
    GREEN = auto()
```

```
    BLUE = auto()
```

```
def describe(color):
```

```
    match color:
```

```
        case Color.RED:
```

```
            return "Red like fire"
```

```
        case Color.GREEN:
```

```
            return "Green like forest"
```

```
        case _:
```

```
            return "Other color"
```


比 **switch** 强的地方：1. 匹配结构，不仅仅是数值 2. 支持 guard (if 条件) 3. 自动解构赋值 4. | 连接多个模式

4. 场景

场景 1：for-else 替代哨兵变量

```
# 查快递状态
tracking_numbers = ["SF123", "SF456", "SF789"]

for tn in tracking_numbers:
    if query_status(tn) == "delivered":
        print(f"Found delivered: {tn}")
        break
else:
    print("No delivered packages found")
```

场景 2：海象在解析中的应用

```
import re

# 多次正则提取
def parse_config(text):
    config = {}
    # 解析 key=value 行
    pattern = re.compile(r"(\w+)\s*=\s*(.*)")

    for line in text.splitlines():
        if m := pattern.match(line.strip()):
            key, value = m.groups()
            config[key] = value
            print(f"  Parsed: {key} = {value}")

    return config
```

场景 3：match-case 的 JSON 路由

```
def handle_event(event):
    """处理各种事件——比 if-elif 干净得多"""
    match event:
        case {"type": "login", "user": user}:
            return handle_login(user)

        case {"type": "logout", "user": user, "session": session}:
            return handle_logout(user, session)

        case {"type": "error", "code": code, "message": msg}:
            return handle_error(code, msg)

        case {"type": "message", "text": text, "reply_to": reply_to}:
            return handle_reply(text, reply_to)

        case {"type": "message", "text": text}:
            return handle_message(text)

        case _:
            raise ValueError(f"Unknown event: {event}")
```

5. 替代方案对比

场景	推荐	不推荐
找东西找到就停	for-else	flag 变量
while 读行	while line := f.readline():	while True: ... if not line: break
if 中使用赋值+判断	if (n := len(x)) > 0:	n = len(x) + if n > 0:
值匹配	match-case	if-elif-elif...
结构匹配	match-case	isinstance + 手动解包

6. 常见坑

坑 1: for-else 的 break 判断

```
def find_item(items, target):
    for i, item in enumerate(items):
        if item == target:
            print(f"Found at {i}")
            break
    else:
        print("Not found")
        return None
    return i
```

注意: 如果是函数中间 return 而不是 break, else 也会执行!

```
for i in range(3):
    if i == 1:
        return # 从函数返回, 不是 break
```

```
else:
    print("This runs!") # ✗ 因为没 break
```

所以 for-else 只适用 break 场景, 不适用 return

坑 2: 海象操作符的优先级

✗ if n := len(x) > 0: # 等价于 n := (len(x) > 0) → n=True/False

if (n := len(x)) > 0: # ✓ 正确

坑 3: match-case 的_放在最后

```
match code:
    case 200: ...
    case _: ...
    case 404: ... # ✗ Unreachable
```

坑 4: match-case 不是 switch—case 后不能跟任意表达式

x = 10

match 1:

```
case x: ... # 这不匹配 x 的值! 这是赋值给了 x
# 要用 case _ if x == 1: ... 才行
```

7. 代码验证

验证海象操作符消除重复

```
import re
import time
```

data = "error" * 10000

```
def without_walrus():
    results = []
    word = ""
    for ch in data:
        if ch == 'e':
            word = "error"
        if len(word) > 0:
```

```

        results.append(word)
    return results

def with_walrus():
    results = []
    word = ""
    for ch in data:
        if ch == 'e':
            word = "error"
        if (n := len(word)) > 0:
            results.append((n, word))
    return results

```

六、函数

1. 是什么

Python 的函数比其他语言灵活得多——参数系统极其丰富，且函数本身是一等公民（一切皆对象）。

2. 解决了什么问题

- 参数系统的灵活性让 API 设计更优雅——调用方能选择传位置参数还是关键字参数
- `*args` / `**kwargs` 解决了“函数要支持任意数量参数”的问题
- 闭包解决了“函数需要携带上下文”的问题（不用定义一个类）
- 可变默认值陷阱是一个经典测试题——理解它才算理解 Python 对象模型

3. 核心理论

参数系统

```

def complex_func(
    a,                # 位置参数（必填）
    /,                # 分隔符：前面只能位置传参（3.8+）
    b,                # 位置或关键字参数
    *,                # 分隔符：后面只能关键字传参
    c,                # 仅关键字参数
    d=42,             # 默认值参数（可选，在 * 后面也仅限关键字）
    **kwargs          # 关键字参数收集
):
    print(a, b, c, d, kwargs)

# 正确调用
complex_func(1, 2, c=3, d=4, extra="hello")
# 1 2 3 4 {'extra': 'hello'}

# 错误调用
# complex_func(1, 2, 3, 4, extra="hello") # ✗ c 不能位置传
# complex_func(a=1, b=2, c=3)             # ✗ a 不能关键字传

```

为什么需要 / 和 *： - / 前面的参数只能位置传——这是为了以后可以改参数名而不影响已有代码 - * 后面的参数只能关键字传——避免参数太多时搞错顺序

可变参数

```

# *args — 任意数量位置参数
def sum_all(*numbers):
    return sum(numbers)

```

```

print(sum_all(1, 2, 3))          # 6
print(sum_all(1, 2, 3, 4, 5))   # 15

# 解包调用
sum_all(*[1, 2, 3])  # 6 — 列表展开传入

# **kwargs — 任意数量关键字参数
def build_url(base, **params):
    query = "&".join(f"{k}={v}" for k, v in params.items())
    return f"{base}?{query}"

print(build_url("http://api.com", name="Leo", age=30))
# http://api.com?name=Leo&age=30

# 解包调用
build_url("http://api.com", **{"name": "Leo", "age": 30})

```

可变默认值陷阱

```

# ✘ 经典大坑
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1))      # [1]
print(add_item(2))      # [1, 2]——不是 [2]!
print(add_item(3))      # [1, 2, 3]

# 原因：默认值在函数定义时求值一次，之后每次都改同一个列表对象
print(add_item.__defaults__) # ([1, 2, 3],)

# ✔ 正确做法
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item(1)) # [1]
print(add_item(2)) # [2]

```

闭包

```

def make_counter(start=0):
    count = start

    def increment(step=1):
        nonlocal count # nonlocal 是关键! 不能省
        count += step
        return count

    def reset():
        nonlocal count
        count = start

    return increment, reset

inc, reset = make_counter(10)
print(inc())      # 11
print(inc(5))     # 16
reset()
print(inc())      # 11

```

闭包的核心：内部函数“记住”了外部函数的变量，即使外部函数已经返回。

lambda

```
# 单行匿名函数
square = lambda x: x ** 2
print(square(5)) # 25

# 常用搭配
pairs = [(1, "b"), (2, "a"), (3, "c")]
pairs.sort(key=lambda x: x[1]) # 按第二个元素排序

sorted([-3, 1, -2, 4], key=lambda x: abs(x))
# [1, -2, -3, 4]

# 限制：只能写表达式，不能写语句
# lambda x: x + 1 # ✓
# lambda x: if x > 0: return x # ✗
# lambda x: x if x > 0 else 0 # ✓ 三元表达式可以
```

4. 场景

场景 1：参数校验 + 关键字参数

```
def create_user(
    /, # username 只能位置传
    username: str,
    *, # 以下必须关键字传
    email: str,
    age: int | None = None,
    is_admin: bool = False,
):
    """创建用户——明确区分必填和可选"""
    if not username.isalnum():
        raise ValueError("Username must be alphanumeric")

    return {
        "username": username,
        "email": email,
        "age": age,
        "is_admin": is_admin,
    }

# 调用
create_user("Leo", email="leo@example.com")
# create_user(username="Leo", email="...") # ✗ username 不能关键字传
```

场景 2：可变参数装饰器

```
import time
from functools import wraps

def timed(reps=1):
    """测量函数执行时间的装饰器"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            total = 0
            for _ in range(reps):
                start = time.perf_counter()
                result = func(*args, **kwargs)
```

```

        total += time.perf_counter() - start
    print(f"{func.__name__}: avg {total/rep:.6f}s")
    return result
    return wrapper
return decorator

```

```

@timed(reps=10)
def slow_function():
    time.sleep(0.01)

```

```
slow_function() # slow_function: avg 0.0100xxxxs
```

5. 替代方案对比

需求	Python	其他语言	说明
可选参数	默认值	重载	Python 简洁
任意数量参数	*args	可变参数/C++initializer_list	差不多
关键字参数	**kwargs	无/命名参数	Python 独家
仅位置参数	/	传统	3.8+
仅关键字参数	*	无	Python 独家
匿名函数	lambda	箭头函数	其他语言广泛

6. 常见坑

```

# 坑 1: lambda 闭包延迟绑定
funcs = [lambda: i for i in range(5)]
print([f() for f in funcs]) # [4, 4, 4, 4, 4]! 不是 [0,1,2,3,4]

```

```

# 原因: lambda 中的 i 是引用, 循环结束时 i=4
# ✓ 默认参数绑定当前值
funcs = [lambda i=i: i for i in range(5)]
print([f() for f in funcs]) # [0, 1, 2, 3, 4]

```

```

# 坑 2: 默认值引用同一个可变对象
# 参见上面的可变默认值陷阱

```

```

# 坑 3: *args 和 **kwargs 混合时**kwargs 必须在最后
# def f(**kwargs, *args): # ✗ SyntaxError

```

```

# 坑 4: nonlocal 只向上查找最近的非全局作用域
x = "global"

```

```

def outer():
    x = "outer"
    def inner():
        x = "inner"
        def innermost():
            nonlocal x # 这是 inner 的 x, 不是 outer 的
            x = "changed"
        innermost()
    print(x) # "inner"—outer 的 x 没变

```

7. 代码验证

```

# 验证默认值在定义时创建
import time

```

```

def get_default_list(value):
    """返回新列表"""

```

```

    return [value]

def get_default_list_bad(
    value,
    items=[time.time()], # 定义时创建，后续调用都用同一个
):
    items.append(value)
    return items

print(get_default_list_bad(1)) # [时间戳, 1]
print(get_default_list_bad(2)) # [时间戳, 1, 2]

# 验证函数对象的 __defaults__
print(get_default_list_bad.__defaults__)
# 显示同一个列表对象

# 验证 nonlocal 作用域链
def make_counters():
    counters = {"a": 0, "b": 0}

    def inc_a():
        counters["a"] += 1 # 不需要 nonlocal!
        return counters["a"]

    def inc_b():
        counters["b"] += 1
        return counters["b"]

    # 为什么这里不用 nonlocal?
    # 因为 counters 是 dict——我们修改的是内容，不是重新赋值
    # nonlocal 只在重新绑定变量名时（=赋值）需要

    return inc_a, inc_b

inc_a, inc_b = make_counters()
print(inc_a()) # 1
print(inc_a()) # 2
print(inc_b()) # 1

```

七、异常处理

1. 是什么

Python 的异常处理采用 **EAFP**（Easier to Ask Forgiveness than Permission）哲学——先试着做，出错了再处理。这和 Java/C 的 **LBYL**（Look Before You Leap）形成对比。

2. 解决了什么问题

```

# LBYL（其他语言风格）
def divide_list(values, divisor):
    if divisor == 0:
        return None
    results = []
    for v in values:
        if isinstance(v, (int, float)):
            results.append(v / divisor)
        else:
            results.append(None)
    return results

# EAFP（Python 风格）

```

```
def divide_list(values, divisor):
    results = []
    for v in values:
        try:
            results.append(v / divisor)
        except (ZeroDivisionError, TypeError):
            results.append(None)
    return results
```

EAFP 的优势：- 代码更短——不用重复检查条件 - 没有竞态条件——检查和操作之间不会有其他线程改变状态 - 异常路径和正常路径分离

3. 核心理论

```
# 完整结构
try:
    result = risky_operation()
except ValueError as e:          # 捕获特定异常
    print(f"Value error: {e}")
except (IOError, OSError) as e:  # 捕获多个异常
    print(f"IO error: {e}")
except Exception:               # 捕获所有（少用）
    print("Something wrong")
    raise                        # 重新抛出
else:
    print(f"Success: {result}")  # 没异常才执行
finally:
    cleanup()                   # 无论是否有异常都执行
```

else 的妙用：把“成功处理逻辑”放在 else 中，而不是在 try 块末尾——这样不会意外捕获到成功处理逻辑中的异常。

自定义异常

```
class AppError(Exception):
    """应用异常基类"""
    pass

class ValidationError(AppError):
    def __init__(self, field: str, message: str):
        self.field = field
        self.message = message
        super().__init__(f"{field}: {message}")

class NotFoundError(AppError):
    def __init__(self, resource: str, id: int):
        self.resource = resource
        self.id = id
        super().__init__(f"{resource} #{id} not found")

# 使用
raise ValidationError("email", "Invalid format")
raise NotFoundError("User", 42)
```

异常链

```
def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError as e:
```



```

        raise AppError(f"Config not found: {path}") from e
    except json.JSONDecodeError as e:
        raise AppError(f"Invalid JSON in {path}") from e

# "from e" 保留了原始异常信息
try:
    load_config("missing.json")
except AppError:
    import traceback
    traceback.print_exc()
    # 显示完整的异常链: FileNotFoundError → AppError

```

4. 场景

场景 1：异常嵌套的层级

```

# 多层函数调用时，在最上层统一处理
class ServiceError(Exception):
    pass

def db_query(sql):
    # 最底层（原始异常）
    raise ConnectionError("DB connection refused")

def get_user(user_id):
    try:
        return db_query(f"SELECT * FROM users WHERE id={user_id}")
    except ConnectionError as e:
        raise ServiceError("Database unavailable") from e

def handler():
    # 最上层（统一处理）
    try:
        user = get_user(42)
        return {"status": "ok", "data": user}
    except ServiceError as e:
        return {"status": "error", "message": str(e)}

```

场景 2：with contextlib.suppress 优雅忽略

```

from contextlib import suppress

# 不用 suppress:
try:
    os.remove("temp.txt")
except FileNotFoundError:
    pass

# 用 suppress（更简洁）:
with suppress(FileNotFoundError):
    os.remove("temp.txt")

# 多个异常
with suppress(FileNotFoundError, PermissionError):
    os.remove("protected.txt")

```

5. 替代方案对比

模式	Python 推荐	适用场景
先检查再做	LBYL	简单条件判断，内部操作

模式	Python 推荐	适用场景
先试后处理	EAFP	IO、网络、用户输入
忽略特定异常	suppress	清理操作（删除不存在的文件）
重新抛出	raise	需要记录日志但不想处理
异常链	raise ... from e	转换异常种类时

6. 常见坑

```
# 坑 1: except 的顺序
try:
    process()
except Exception:      # 先捕获所有
    print("caught")
except ValueError:     # 永远不会到这里
    print("never")

# ✓ 先具体后通用
try:
    process()
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
except Exception:
    print("Other error")

# 坑 2: except: 没指定会捕获 SystemExit 和 KeyboardInterrupt
# 永远不要裸 except:
try:
    user_input()
except:                # 会吞掉 Ctrl+C!
    pass

# ✓ 至少写 except Exception:
try:
    user_input()
except Exception:
    pass              # Ctrl+C 仍然工作

# 坑 3: finally 的 return 会覆盖其他 return
def f():
    try:
        return "try"
    finally:
        return "finally" # ✗ 覆盖了 try 的 return

print(f()) # "finally"

# 坑 4: 空 except 不推荐——你永远不知道什么异常
```

7. 代码验证

```
# 验证 else 的用途
try:
    result = int("42")
except ValueError:
    print("Invalid number")
else:
    # 只有成功才执行
    print(f"Result is {result}")
    # 如果这里写在 try 里不小心抛异常，也会被上面的 except 捕获
```

八、面向对象（OOP）基础

1. 是什么

面向对象编程（OOP）是一种将数据和操作数据的方法组织成“对象”，每个对象包含数据（属性）和方法（行为）。

Python 的 OOP 比大多数语言更灵活——它支持多重继承、方法重写、鸭子类型，以及一切皆对象的哲学。

基础语法速览：

```
class Dog:
    species = "Canis familiaris" # 类属性（所有实例共享）

    def __init__(self, name, age):
        self.name = name # 实例属性
        self.age = age

    def bark(self):
        return f"{self.name} says woof!"

    def __str__(self):
        return f"{self.name} ({self.age}yo)"

buddy = Dog("Buddy", 3)
print(buddy.bark()) # "Buddy says woof!"
print(buddy.species) # "Canis familiaris"
print(buddy) # "Buddy (3yo)"
```

关键点：- `self` 是实例本身的引用（类似其他语言的 `this`），**必须作为第一个参数** - `__init__` 是构造方法（初始化实例），不是构造函数（对象已经创建了再初始化） - `__str__` 是魔术方法，控制 `print(obj)` 的输出 - 实例方法、类方法、静态方法有不同的第一个参数约定

2. 解决了什么问题

问题 1：数据和操作分离

没有 OOP 时，操作数据的函数和数据是分开的：

```
# 函数式风格
def create_dog(name, age):
    return {"name": name, "age": age}

def bark(dog):
    return f"{dog['name']} says woof!"

def celebrate_birthday(dog):
    dog['age'] += 1
```

每增加一个操作，就要加一个新函数。谁创建了数据谁负责操作——没有自然的分组。

OOP 把数据和操作封装在一起——“狗的年龄增加”不再是外部函数的职责，而是 `Dog` 自己的方法。

问题 2：代码复用

直接复制粘贴来修改功能会导致大量重复代码。OOP 的继承允许你在一个地方修改，所有子类自动获得。

问题 3：现实世界建模

OOP 的”对象”和现实世界的”事物”有自然对应关系：用户、订单、商品、请求.....这让代码结构更容易理解。

3. 核心理论

3.1 类属性 vs 实例属性

```
class Employee:
    # 类属性——所有实例共享
    company = "Acme Inc"
    raise_rate = 1.05

    def __init__(self, name, salary):
        # 实例属性——每个实例独有
        self.name = name
        self.salary = salary

    def apply_raise(self):
        self.salary = int(self.salary * self.raise_rate)

e1 = Employee("Alice", 50000)
e2 = Employee("Bob", 60000)

print(e1.company) # "Acme Inc"
print(e2.company) # "Acme Inc"

# 修改类属性影响所有实例
Employee.raise_rate = 1.10
e1.apply_raise()
print(e1.salary) # 55000

# 修改实例属性不影响类
Employee.company = "New Corp"
print(e1.company) # "New Corp"
print(e2.company) # "New Corp"

# 但给实例设置同名属性会屏蔽类属性
e1.company = "Private Co"
print(e1.company) # "Private Co" - 实例属性覆盖
print(e2.company) # "New Corp" - 还是类属性
```

属性查找顺序：实例属性 → 类属性 → 父类（MRO 顺序）

3.2 实例方法、类方法、静态方法

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    # 实例方法——需要访问实例
    def format(self):
        return f"{self.year}-{self.month:02d}-{self.day:02d}"
```

```

# 类方法—需要访问类（第一个参数是 cls，不是 self）
@classmethod
def from_string(cls, date_str):
    """替代构造器：由字符串创建实例"""
    year, month, day = map(int, date_str.split("-"))
    return cls(year, month, day)

# 静态方法—不需要访问类或实例（就是个普通函数放在类里）
@staticmethod
def is_valid(date_str):
    """验证日期字符串格式"""
    try:
        parts = date_str.split("-")
        return len(parts) == 3
    except Exception:
        return False

```

```

# 实例方法：通过实例调用
d = Date(2026, 5, 23)
print(d.format()) # "2026-05-23"

```

```

# 类方法：通过类或实例调用
d2 = Date.from_string("2026-06-01")
print(d2.format()) # "2026-06-01"

```

```

# 静态方法：通过类或实例调用
print(Date.is_valid("2026-05-23")) # True

```

方法类型	第一个参数	能访问实例	能访问类	什么时候用
实例方法	self	✓	✓	绝大多数方法
类方法	cls	✗	✓	工厂方法、替代构造器
静态方法	无	✗	✗	工具函数，语义上属于这个类

3.3 继承

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclasses must implement")

    def __str__(self):
        return f"Animal: {self.name}"

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

    def fetch(self):
        return f"{self.name} fetches the ball"

class Cat(Animal):
    def __init__(self, name, lives=9):
        super().__init__(name) # 调用父类构造
        self.lives = lives

    def speak(self):
        return f"{self.name} says Meow!"

```

```

def __str__(self):
    return f"Cat: {self.name} ({self.lives} lives left)"

class RobotDog(Dog):
    """多重继承层次——覆盖和不覆盖"""
    def speak(self):
        return f"{self.name} says Beep beep!"

animals = [Dog("Buddy"), Cat("Whiskers"), RobotDog("R2D2")]
for a in animals:
    print(a.speak()) # 多态：不同类，同一接口
# Buddy says Woof!
# Whiskers says Meow!
# R2D2 says Beep beep!

print(isinstance(animals[0], Animal)) # True
print(isinstance(animals[2], Dog))    # True (继承链)
print(issubclass(RobotDog, Animal))   # True

```

super() 的作用：调用父类方法。在 `__init__` 中调用 `super().__init__()` 是最常见的模式——确保父类的初始化逻辑被执行。

3.4 魔术方法 (Magic Methods / Dunder Methods)

Python 的类可以通过实现特定名称的方法来参与语言的内建操作：

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        """开发者看到的表示"""
        return f"Vector({self.x}, {self.y})"

    def __str__(self):
        """用户看到的表示"""
        return f"({self.x}, {self.y})"

    def __add__(self, other):
        """支持 + 运算符"""
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        """支持 - 运算符"""
        return Vector(self.x - other.x, self.y - other.y)

    def __eq__(self, other):
        """支持 == 比较"""
        return self.x == other.x and self.y == other.y

    def __abs__(self):
        """支持 abs()"""
        return (self.x**2 + self.y**2) ** 0.5

    def __bool__(self):
        """支持布尔判断"""
        return self.x != 0 or self.y != 0

```

```

v1 = Vector(3, 4)
v2 = Vector(1, 2)

print(v1)           # (3, 4) ← __str__
print(repr(v1))     # Vector(3, 4) ← __repr__
print(v1 + v2)      # (4, 6) ← __add__
print(v1 - v2)      # (2, 2) ← __sub__
print(v1 == Vector(3, 4)) # True ← __eq__
print(abs(v1))      # 5.0 ← __abs__
print(bool(Vector(0, 0))) # False ← __bool__

```

常用魔术方法速查：

类别	方法	触发时机
构造	<code>__init__</code> , <code>__new__</code>	创建对象
字符串	<code>__str__</code> , <code>__repr__</code> , <code>__format__</code>	<code>print</code> , <code>str</code> , <code>f-string</code>
算术	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code>	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>
比较	<code>__eq__</code> , <code>__lt__</code> , <code>__gt__</code> , <code>__le__</code> , <code>__ge__</code>	<code>==</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
容器	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__contains__</code>	<code>len</code> , <code>[]</code> , <code>in</code>
迭代	<code>__iter__</code> , <code>__next__</code>	<code>for</code> 循环
可调	<code>__call__</code>	<code>obj()</code>
上下文	<code>__enter__</code> , <code>__exit__</code>	<code>with</code> 语句

3.5 property — 控制属性访问

```

class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius # 约定：_ 前缀表示"内部使用"

    @property
    def celsius(self):
        """只读属性（但没有 setter 就无法赋值）"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """setter：赋值时做校验"""
        if value < -273.15:
            raise ValueError("Temperature below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):
        """计算属性（只读）"""
        return self._celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5 / 9

# 使用
t = Temperature(25)
print(t.celsius)      # 25
t.celsius = 30        # ✓ 调用 setter

```

```

print(t.fahrenheit)          # 86.0
# t.celsius = -300          # ✗ ValueError
t.fahrenheit = 100          # ✓ 设置 fahrenheit 连带着改 celsius
print(t.celsius)            # 37.78

```

property 的价值： 让你先写简单的属性访问，后续再添加校验/计算逻辑而不改变调用方的代码。

4. 场景

场景 1：数据验证类

```

class User:
    """带验证的用户类"""
    def __init__(self, username, email, age=None):
        self.username = username # 触发 setter
        self.email = email
        self.age = age

    @property
    def username(self):
        return self._username

    @username.setter
    def username(self, value):
        if not isinstance(value, str) or len(value) < 3:
            raise ValueError("Username must be at least 3 characters")
        if not value.isalnum():
            raise ValueError("Username must be alphanumeric")
        self._username = value

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, value):
        if "@" not in value:
            raise ValueError("Invalid email address")
        self._email = value

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value is not None and (value < 0 or value > 150):
            raise ValueError("Age must be between 0 and 150")
        self._age = value

# 使用
u = User("Leo", "leo@example.com", 30) # ✓
# u2 = User("L", "leo@.com")           # ✗ ValueError

```

场景 2：链式 API（类似 pandas 风格）

```

class Query:
    def __init__(self, data):
        self._data = list(data)

    def filter(self, predicate):

```



```

    self._data = [x for x in self._data if predicate(x)]
    return self # 返回 self 实现链式调用

def map(self, func):
    self._data = [func(x) for x in self._data]
    return self

def sort(self, key=None, reverse=False):
    self._data = sorted(self._data, key=key, reverse=reverse)
    return self

def limit(self, n):
    self._data = self._data[:n]
    return self

def to_list(self):
    return self._data

# 链式使用
result = (
    Query([1, 2, 3, 4, 5, 6])
    .filter(lambda x: x > 2)
    .map(lambda x: x ** 2)
    .sort(reverse=True)
    .limit(2)
    .to_list()
)
print(result) # [36, 25]

```

5. 替代方案对比

场景	OOP	替代方案
简单数据容器	class + <code>init</code>	dataclass / namedtuple
带验证的属性	@property	手动 getter/setter
单方法行为	class + 方法	闭包 (closure)
多种类型行为	继承 + 多态	singledispatch
简单命名元组	namedtuple / dataclass	class 太啰嗦

什么时候用 **class**，什么时候不用：- 有内部状态需要维护 → class - 只有行为没有状态 → 用函数 - 只有数据没有行为 → 用 dataclass / namedtuple - 需要多种实现的同一接口 → 用 ABC / Protocol

6. 常见坑

```

# 坑 1: 可变默认值 (跟函数一样的问题!)
class Bad:
    def __init__(self, items=[]): # ✗ 所有实例共享同一个列表
        self.items = items

a = Bad()
b = Bad()
a.items.append(1)
print(b.items) # [1] - 被 a 污染了

class Good:
    def __init__(self, items=None):
        self.items = items if items is not None else []

# 坑 2: 双下划线属性不是私有——是名称修饰

```

```

class Secret:
    def __init__(self):
        self.__secret = 42

s = Secret()
# print(s.__secret)      # ✗ AttributeError
print(s._Secret__secret) # 42 — 仍然能访问!
# Python 没有真正的私有属性, __ 只是避免子类意外覆盖
# 单下划线 _ 是约定—"这是内部实现, 不要碰"

# 坑 3: 继承时忘记调用 super().__init__()
class Parent:
    def __init__(self):
        self.initialized = True

class Child(Parent):
    def __init__(self):
        pass # 忘记调用 super().__init__()

c = Child()
# print(c.initialized) # ✗ AttributeError — Parent.__init__ 没执行

# 坑 4: @staticmethod 和 @classmethod 的误用
# 如果方法里用到了 self, 永远不要用 @staticmethod
class MathUtils:
    @staticmethod
    def add(a, b): # ✓ 不需要访问类或实例—纯工具
        return a + b

    @classmethod
    def from_polar(cls, r, theta): # ✓ 需要返回 cls 实例
        return cls(r * cos(theta), r * sin(theta))

# 坑 5: 多重继承的 MRO (方法解析顺序)
class A:
    def method(self):
        return "A"

class B(A):
    def method(self):
        return "B"

class C(A):
    def method(self):
        return "C"

class D(B, C):
    pass

d = D()
print(d.method()) # "B" — C3 线性化算法决定
print(D.__mro__)  # D → B → C → A → object

```

7. 代码验证

```

# 验证 isinstance 和 type 的区别
class Parent:
    pass

class Child(Parent):
    pass

c = Child()

```

```

print(type(c) is Child)    # True
print(type(c) is Parent)  # False — type 精确匹配

print(isinstance(c, Child)) # True
print(isinstance(c, Parent)) # True — isinstance 走继承链

# 验证 property 的延迟计算
class Lazy:
    @property
    def expensive(self):
        """每次访问都重新计算"""
        import time
        time.sleep(1)
        print("Computing...")
        return 42

    @cached_property
    def cached(self):
        """只计算一次，之后缓存"""
        import time
        time.sleep(1)
        print("Caching...")
        return 99

from functools import cached_property
l = Lazy()
print(l.cached) # 第一次：计算
print(l.cached) # 第二次：直接返回缓存值

```

九、文件 I/O 与 pathlib

1. 是什么

Python 处理文件有两种主要方式：传统的 `open()` + `with` 语句，以及现代的 `pathlib.Path` 对象（Python 3.4+）。

2. 解决了什么问题

- 传统文件操作需要 `open/read/write/close` 手动管理生命期
- `os.path` 的函数式 API（`os.path.join`, `os.path.exists`）不 OOP，难链式调用
- 跨平台路径分隔符（/ vs \）需要手动处理
- `pathlib` 用对象方法替代了零散的 `os.path` 函数

3. 核心理论

with 语句（上下文管理器）

```

# 不需要手动 close—with 退出时自动关闭
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()      # 整个文件
    lines = f.readlines()   # 按行列表

# 大文件逐行读取（不加载全部到内存）
with open("large.log", "r") as f:
    for line in f:           # 惰性读取
        process(line)

```

```
# 写文件
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello, World!\n")
    f.writelines(["line1\n", "line2\n"])
```

打开模式速查：

模式	读/写	指针位置	文件存在	文件不存在
"r"	读	开头	✓	✗ 报错
"r+"	读写	开头	✓	✗ 报错
"w"	写	开头	✗ 清空	✓ 创建
"w+"	读写	开头	✗ 清空	✓ 创建
"a"	写	末尾	✓ 追加	✓ 创建
"a+"	读写	末尾	✓ 追加	✓ 创建
"x"	写	开头	✗ 报错	✓ 创建 (排他)

pathlib (推荐)

```
from pathlib import Path

# 路径操作
p = Path("data/config.json")
p.parent      # data/
p.stem        # config
p.suffix       # .json
p.name         # config.json

# 检查
p.exists()      # True/False
p.is_file()
p.is_dir()

# 读写文本
p.read_text()      # 整个文件为字符串
p.write_text("hello") # 写入字符串

# 读写字节
p.read_bytes()      # bytes
p.write_bytes(b"\x00\x01") # 写入

# 遍历
for py_file in Path("src").rglob("*.py"):
    print(py_file)

# 创建目录
Path("data/logs").mkdir(parents=True, exist_ok=True)

# 重命名/移动
Path("old.txt").rename("new.txt")

# 路径拼接 (重载了 / 运算符)
data_dir = Path("data")
log = data_dir / "logs" / "app.log"
```

4. 场景

场景 1：递归查找并处理文件

```

from pathlib import Path

def find_and_process(root, pattern, callback):
    """在 root 下递归查找匹配 pattern 的文件，对每个文件调用 callback"""
    root = Path(root)
    for filepath in root.rglob(pattern):
        if filepath.is_file():
            callback(filepath)

# 统计所有 .py 文件的行数
total_lines = 0
def count_lines(path):
    global total_lines
    content = path.read_text()
    lines = content.count("\n") + 1
    total_lines += lines
    print(f"{path}: {lines} lines")

find_and_process(".", "*.py", count_lines)
print(f"Total: {total_lines} lines")

```

场景 2：安全写入（防止写一半崩溃）

```

from pathlib import Path
import tempfile
import os

def safe_write(path: str, content: str):
    """先写到临时文件，再原子替换"""
    path = Path(path)
    # 在同一个目录下创建临时文件（保证跨设备）
    with tempfile.NamedTemporaryFile(
        mode="w",
        dir=path.parent,
        suffix=".tmp",
        delete=False,
    ) as tmp:
        tmp.write(content)
        tmp_path = tmp.name

    # 原子替换
    os.replace(tmp_path, path)

# 使用
safe_write("config.json", '{"version": 2}')

```

场景 3：内存文件（StringIO / BytesIO）

```

from io import StringIO, BytesIO

# 字符串当作文件操作（测试时超有用）
buffer = StringIO()
buffer.write("Hello\n")
buffer.write("World\n")

buffer.seek(0)
print(buffer.read()) # "Hello\nWorld\n"

# 字节缓冲
bio = BytesIO()
bio.write(b"\x00\x01\x02")
bio.seek(0)
data = bio.read()

```

5. 替代方案对比

操作	旧方式 (os.path)	新方式 (pathlib)
路径拼接	os.path.join("a", "b")	Path("a") / "b"
文件名	os.path.basename(p)	Path(p).name
扩展名	os.path.splitext(p)[1]	Path(p).suffix
文件存在	os.path.exists(p)	Path(p).exists()
创建目录	os.makedirs(p, exist_ok=True)	Path(p).mkdir(parents=True)
遍历	os.walk(root)	Path(root).rglob("*")

推荐：新代码全部用 pathlib。

6. 常见坑

```
# 坑 1: 文件打开后忘记 close
f = open("data.txt")
data = f.read()
# 没有 close! 如果在 with 块中抛出异常, 文件可能不会关闭
```

```
# 当然用 with 就不会有这个问题
with open("data.txt") as f:
    data = f.read()
# 自动关闭
```

```
# 坑 2: pathlib 的 / 运算符返回 Path, 不是 str
p = Path("data") / "file.json"
# 需要 str 的地方要显式转换
# json.load(open(p))      # ✓ 有些函数接受 Path
# os.system(f"cat {p}")    # ✗ TypeError
os.system(f"cat {str(p)}") # ✓
```

```
# 坑 3: 文本模式 vs 二进制模式
# Windows 上 "r" 模式会转换 \r\n → \n
# 处理图片/压缩包要用二进制模式
with open("image.jpg", "rb") as f:
    data = f.read() # bytes, 不会被转换
```

7. 代码验证

```
# 验证 pathlib / 运算符是纯路径操作 (不访问磁盘)
from pathlib import Path

p = Path("/nonexistent/directory") / "file.txt"
print(p) # /nonexistent/directory/file.txt
# 不会报错——路径只是字符串操作

# verify 存在检查才访问磁盘
print(p.exists()) # False — 这次才真的检查磁盘
```

十、模块与 import

1. 是什么

Python 的模块系统让代码可以按文件组织, 并通过 import 跨文件复用。每个 .py 文件就是一个模块。

2. 解决了什么问题

- 代码拆分到不同文件，避免一个文件几千行
- 命名空间隔离——`module.function()` 不会冲突
- 复用已有的库——标准库和第三方包

3. 核心理论

```
# 三种 import 方式
import os                # os 命名空间可用，用 os.path.join()
from pathlib import Path  # Path 直接可用
from datetime import datetime as dt  # 别名

# 绝对 vs 相对导入
# ✓ 推荐绝对导入
from my_package.utils.helpers import parse_date

# ✗ 相对导入容易混淆
# from ..utils.helpers import parse_date

# 模块也是对象
import math
print(math.__name__)  # "math"
print(math.pi)        # 3.14159...
```

if name == "main"

```
# 当文件被直接运行时 __name__ = "__main__"
# 当文件被 import 时 __name__ = 模块名
def main():
    print("Running directly")

if __name__ == "__main__":
    main()
```

这个模式解决了“可执行/可导入”的双重需求：- python script.py → 执行 main() - import script → 不执行 main()

模块搜索路径

```
import sys
print(sys.path)
# ['', '/usr/lib/python3.x', '/usr/local/lib/python3.x/site-packages', ...]

# 可以动态添加
sys.path.append("/my/custom/path")

# 但推荐用 PYTHONPATH 环境变量或安装到 site-packages
```

4. 场景

场景 1：项目的标准包结构

```
myproject/
├── pyproject.toml      # 项目元数据和依赖
├── src/
│   └── myproject/
│       ├── __init__.py  # 包初始化
```

```

├── main.py          # 入口
├── config.py        # 配置
├── utils/
│   ├── __init__.py
│   └── helpers.py
└── tests/
    ├── test_main.py
    └── test_helpers.py

```

```

# src/myproject/main.py
from myproject.config import load_config
from myproject.utils.helpers import format_date

```

```

def main():
    config = load_config()
    print(format_date(config["start_date"]))

```

```

if __name__ == "__main__":
    main()

```

5. 常见坑

```

# 坑 1: 循环导入
# a.py: from b import foo
# b.py: from a import bar
# 运行时会报 ImportError

```

```

# 解决方案:
# 1. 把公用的放第三个文件
# 2. 延迟导入 (在函数内 import)
# 3. 重构结构消除循环依赖

```

```

# 坑 2: import * 污染命名空间
from os import * # ✗ 不推荐
# 你不知道导入了什么, 可能会覆盖已有变量

```

```

from pathlib import Path # ✔ 明确

```

```

# 坑 3: __init__.py 为空时, import package 不会自动导入子模块
# 需要在 __init__.py 显式 import
# __init__.py
# from . import config
# from . import utils

```

```

# 坑 4: 相对导入在 __main__ 中不能用
# 如果你的文件是入口 (__main__), 里面不能用 from . import xxx

```

6. 代码验证

```

# 验证 __name__ 的不同值
# running_as_main.py
print(f"__name__ = {__name__}") # 直接运行 → "__main__"

```

总结

知识点	一句话记法	核心价值
Python 哲学	显式优于隐式, 可读性优先	知道为什么 Python 这样设计
数据类型	int 无限、bool 是 int 子类、None 判断用 is	避开常见类型坑

知识点	一句话记法	核心价值
字符串	用 f-string 格式化，用 join 拼接	性能和可读性双赢
容器	list 最常用、set 去重、dict 关联	选择正确的容器
控制流	for-else 替代哨兵、:= 消除重复、match 替代 if-elif	少写样板代码
函数	参数系统灵活、默认值陷阱要避免	API 设计的核心
OOP	class 封装数据和行为、property 控制访问	数据和操作在一起
异常	EAFP、异常链保留上下文	生产级错误处理
文件 I/O	用 with、用 pathlib	更安全更现代
模块	__name__ == "__main__" 区分入口和库	代码组织的基础

明天预告：Day 2 — 进阶语法 - 迭代器（Iterable vs Iterator、自定义迭代器） - 闭包（Closure、nonlocal） - 装饰器（@语法糖、带参数装饰器、functools.wraps） - 生成器（yield、yield from、生成器表达式） - 上下文管理器（with、contextmanager、enter/exit）

Day 2 — Python 进阶核心概念（完整版）

一、迭代器（Iterator）

1. 是什么

迭代器是一个可以逐个返回元素的对象。它实现了迭代器协议：__iter__() 返回自身，__next__() 返回下一个元素，没有更多元素时抛 StopIteration。

简单说：能用 for x in obj: 遍历的东西，要么本身就是迭代器，要么是可迭代对象（iterable）可以转成迭代器。

2. 解决了什么问题

没有迭代器之前，遍历集合你得手动维护索引：

```
i = 0
while i < len(items):
    print(items[i])
    i += 1
```

问题：- 每种数据结构遍历方式不一样（列表用下标，字典用 key，文件读行） - 不是所有东西都有下标（比如生成器、网络流） - 代码重复，容易下标越界

迭代器协议统一了遍历方式：不管底层是什么数据结构，都用 for x in obj。

3. 核心理论

迭代器协议包含两个方法：

方法**作用**`__iter__(self)`

返回迭代器对象自身。让迭代器也可以用在 for 循环中

`__next__(self)`返回下一个元素。没有元素时抛 `StopIteration``for x in obj` 本质上做的就是：

```
# for x in obj 等价于：
iter_obj = iter(obj)      # 调 obj.__iter__()
while True:
    try:
        x = next(iter_obj) # 调 iter_obj.__next__()
        # 执行循环体
    except StopIteration:
        break              # 没有更多元素，退出
```

区分两个概念：

概念**定义****例子**

可迭代对象 (iterable)

实现了 `__iter__`，返回迭代器 列表、字典、字符串、文件

迭代器 (iterator)

实现了 `__iter__ + __next__` 生成器、`iter()` 返回值

关键区别： - 可迭代对象每次 `iter()` 都返回**新的**迭代器，所以可以多次遍历 - 迭代器只遍历一次就耗尽
 - 所有迭代器也都是可迭代对象（因为它自己也实现了 `__iter__`）

4. 场景**场景 1：自定义可迭代对象**

```
class Countdown:
    """从 n 往下数到 1"""
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        return CountdownIterator(self.n)
```

```
class CountdownIterator:
    def __init__(self, n):
        self.current = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value
```

```
for i in Countdown(5):
    print(i)
# 5 4 3 2 1
```

场景 2：用生成器简化（生成器就是迭代器，下一节细讲）

```
class Countdown:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        # 生成器函数自动实现了迭代器协议
        current = self.n
        while current > 0:
            yield current
            current -= 1

for i in Countdown(5):
    print(i)
# 5 4 3 2 1
```

场景 3：惰性遍历大文件（文件对象本身就是迭代器）

```
with open("large_file.log") as f:
    # f 本身就是迭代器，一次读一行，不把整个文件加载到内存
    for line in f:
        process(line)
```

5. 替代方案对比

方式	内存	统一性	灵活性
迭代器协议	低（惰性）	全部统一 for x in obj	自己控制遍历逻辑
手动下标循环	低	不统一（每种结构写法不同）	只能索引访问
一次性加载全部	高（全在内存）	统一	简单但费内存

6. 常见坑

坑 1：迭代器只能遍历一次

```
nums = [1, 2, 3]
it = iter(nums)
print(list(it))
print(list(it))
# it 是迭代器
# [1, 2, 3]
# [] - 耗尽了

# 而列表本身不是迭代器，可多次遍历
print(list(nums))
print(list(nums))
# [1, 2, 3]
# [1, 2, 3] - 每次 iter() 产生新迭代器
```

因为每次 for x in nums 都会调 iter(nums) 返回一个新的迭代器。

坑 2：在遍历时修改可迭代对象

```
items = [1, 2, 3, 4, 5]
for item in items:
    if item % 2 == 0:
        items.remove(item)
print(items)
# 遍历时修改！元素会移位跳过
# 结果不一定对

# 解决方案：遍历副本
for item in items[:]:
    if item % 2 == 0:
        items.remove(item)
# items[:] 创建副本
```

坑 3：自定义迭代器忘记 __iter__

```

class MyIter:
    def __init__(self, data):
        self.data = data
        self.i = 0

    def __next__(self):
        if self.i >= len(self.data):
            raise StopIteration
        value = self.data[self.i]
        self.i += 1
        return value

it = MyIter([1, 2, 3])
print(next(it))          # 1
print(next(it))          # 2

for x in it:              # TypeError!
    print(x)

```

修复：加 `__iter__`：

```

def __iter__(self):
    return self

```

7. 代码验证

```

class Fibonacci:
    """斐波那契数列迭代器（可指定最大项数）"""
    def __init__(self, max_count):
        self.max_count = max_count
        self.count = 0
        self.a, self.b = 0, 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.max_count:
            raise StopIteration
        value = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1
        return value

for n in Fibonacci(10):
    print(n, end=' ')
# 0 1 1 2 3 5 8 13 21 34

```

迭代器和生成器的关系：**生成器是迭代器的一种便捷实现方式**。所有生成器都是迭代器，但迭代器不一定是生成器。下一节就讲生成器。

二、闭包（Closure）

1. 是什么

闭包是一个**函数 + 它捕获的外部变量**的组合物。简单说：函数内部定义的函数，能记住并访问外部函数的变量，即使外部函数已经执行完毕。

2. 解决了什么问题

在需要“一个函数 + 附带状态”的场景下，闭包让你不用写类就能创建带状态的函数。Python 的函数是一等公民（可以传给别的函数、可以从函数返回），闭包利用这一点实现了轻量级的“函数对象+私有状态”。

如果没有闭包，你只能用：- 全局变量（不安全，容易被意外修改）- 类（太重，有时大材小用）

3. 核心理论

Python 的作用域遵循 LEGB 规则：- Local — 当前函数内 - Enclosing — 外层函数 - Global — 全局 - Built-in — 内置

当内层函数引用外层函数的变量时，Python 会把外层变量“打包”到这个内层函数上，一起返回。只要内层函数还存在，那些被引用的外层变量就不会被垃圾回收。

关键点：**被记住的变量是“活的”**，不是快照。后续修改外层变量的值，内层函数看到的是修改后的值。

4. 场景

场景 1：计数器

```
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c1 = make_counter()
c2 = make_counter()
print(c1()) # 1
print(c1()) # 2
print(c2()) # 1 - c2 有自己的 count
```

场景 2：函数工厂

```
def make_power(n):
    def power(x):
        return x ** n
    return power

square = make_power(2)
cube = make_power(3)
print(square(5)) # 25
print(cube(5))   # 125
```

场景 3：延迟执行 / 回调

```
def make_greeter(greeting):
    def greet(name):
        return f"{greeting}, {name}!"
    return greet

say_hi = make_greeter("你好")
say_hello = make_greeter("Hello")
print(say_hi("小明"))    # 你好, 小明!
print(say_hello("Bob"))  # Hello, Bob!
```

5. 替代方案对比

方案	代码量	状态可见性	额外能力
闭包	少	外部不可直接访问（封装性好）	只能有一个入口（函数调用）
类	多	可以通过属性访问	可以有多个方法
全局变量	最少	所有人都能改（不安全）	到处能用（也是缺点）

什么时候用闭包？只需要一个函数、一段简单的状态、不需要多个方法时，闭包比类更简洁。

什么时候用类？状态复杂、需要多个方法操作、需要继承等 OOP 特性时。

6. 常见坑

坑 1：延迟绑定（经典面试题）

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)

print([f() for f in funcs]) # [2, 2, 2], 不是 [0, 1, 2]
```

因为三个 `f` 引用的是同一个 `i`，循环结束时 `i=2`。

修复方案一：默认参数绑定（绑定的是值）

```
for i in range(3):
    def f(x=i): # 用默认参数把当前 i 的值"冻结"进去
        return x
    funcs.append(f)
```

修复方案二：闭包套闭包

```
def make_f(x):
    return lambda: x

for i in range(3):
    funcs.append(make_f(i))
```

坑 2：忘记 `nonlocal`

```
def outer():
    x = 0
    def inner():
        x += 1 # UnboundLocalError! Python 认为 x 是局部变量
        return x
    return inner
```

需要 `nonlocal x` 告诉 Python：这个 `x` 不是本地的，是外层作用域的。

7. 代码验证

```
# 闭包基础
def make_adder(n):
    def add(x):
        return x + n
```

```

    return add

add5 = make_adder(5)
print(add5(10))    # 15
print(add5(100))   # 105

# nonlocal 修改外部变量
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c = make_counter()
print(c()) # 1
print(c()) # 2
print(c()) # 3

```

三、装饰器 (Decorator)

1. 是什么

装饰器是一个接受函数作为参数、返回新函数的函数。它让你在不修改原函数代码的前提下，给函数添加额外功能。

@decorator 是语法糖，等价于 `func = decorator(func)`。

2. 解决了什么问题

真实项目里经常需要给大量函数加相同的功能：打日志、计耗时、权限校验、缓存、重试逻辑。如果每个函数里都写一遍重复代码，就是严重的 DRY 违反。

装饰器让你把“增强功能”抽出来写一次，然后一行 @ 就能贴到任意函数上。

3. 核心理论

装饰器的本质是三层结构：

```

def decorator(func):          # 第 1 层：接收原函数
    def wrapper(*args, **kwargs): # 第 2 层：包裹函数，接收原函数的参数
        # 调用前做的事
        result = func(*args, **kwargs) # 调用原函数
        # 调用后做的事
        return result
    return wrapper            # 返回包裹函数

```

@decorator 做的事情等价于：

```

def say_hi():
    print("Hi!")

say_hi = decorator(say_hi) # 把 say_hi 替换成 wrapper

```

所以调用 `say_hi()` 时，实际跑的是 `wrapper()`，里面才调了真正的原函数。

4. 场景

场景 1：日志记录

```
import functools

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] 调用 {func.__name__}, 参数={args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"[LOG] {func.__name__} 返回 {result}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(3, 5)
# [LOG] 调用 add, 参数=(3, 5), {}
# [LOG] add 返回 8
```

场景 2：计时

```
import time
import functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} 耗时 {elapsed:.4f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

slow_sum(10_000_000)
# slow_sum 耗时 0.3125s
```

场景 3：重试（真实项目的常见需求）

```
import functools
import time

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"第 {attempt} 次失败: {e}")
                    if attempt == max_attempts:
                        raise
                    time.sleep(delay)
            return None
        return wrapper
    return decorator
```



```

        return wrapper
    return decorator

@retry(max_attempts=3, delay=0.5)
def unstable_api():
    import random
    if random.random() < 0.7:
        raise ConnectionError("网络不稳定")
    return "成功!"

print(unstable_api())

```

5. 替代方案对比

方案	侵入性	复用性	使用成本
装饰器	零（不改原函数）	高	一行@
手动在函数内写重复代码	高	低	每个函数都要写
函数包装器模式	低	中	要手动调用 wrapper
回调/钩子	高（改框架）	高	复杂度高

6. 常见坑

坑 1：忘记 @functools.wraps

```

def decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@decorator
def my_func():
    """做重要的事情"""
    pass

print(my_func.__name__) # 'wrapper', 不是 'my_func'
print(my_func.__doc__) # None, 不是 '做重要的事情'

```

很多调试工具和框架依赖这些元信息，会出问题。

坑 2：多个装饰器的执行顺序

```

@decorator_a
@decorator_b
def my_func():
    pass

# 等价于: my_func = decorator_a(decorator_b(my_func))
# 执行顺序: 先跑 decorator_a 第一次, 再跑 decorator_b 第一次
#           然后跑 my_func, 再跑 decorator_b 收尾, 再跑 decorator_a 收尾

```

两层装饰器像洋葱一样：外层先进入、最后退出。

坑 3：带参数的装饰器需要三层函数

```

# 无参数：两层
@retry
def f(): ...

# 有参数：三层（retry 返回值才是装饰器）

```

```
@retry(max_attempts=3)
def f(): ...
```

7. 代码验证

```
import functools

def validate_positive(func):
    @functools.wraps(func)
    def wrapper(n):
        if n < 0:
            raise ValueError(f"参数必须为正数，收到 {n}")
        return func(n)
    return wrapper

@validate_positive
def sqrt_approx(n):
    return n ** 0.5

print(sqrt_approx(9))    # 3.0
# print(sqrt_approx(-1)) # ValueError
```

四、生成器（Generator）和 yield

1. 是什么

生成器是一个可以暂停和恢复执行的函数。用 `yield` 替代 `return`，函数就变成了生成器。每次调用 `yield`，函数暂停并返回值，下次从暂停处继续。

生成器是迭代器的一种便捷实现。学完迭代器再看生成器，就是“迭代器的自动化版本”。

2. 解决了什么问题

内存问题：当处理大量数据时，一次性全部加载到内存可能不可行。生成器让你“用多少算多少”，一次只生成一个值。

流式处理：数据无限（如实时流、日志流）时，不可能全部存下来。生成器可以不断地产生新数据。

延迟计算：不是每个值都会用到，何必提前算完？生成器惰性求值，用到才算。

3. 核心理论

```
def simple_gen():
    print("开始")
    yield 1
    print("继续")
    yield 2
    print("结束")

g = simple_gen() # 什么都没打印！只是创建了生成器对象
print(next(g))  # 打印"开始", 输出 1
print(next(g))  # 打印"继续", 输出 2
print(next(g))  # 打印"结束", 抛 StopIteration
```

每个 `yield` 像设了一个断点：1. 调用 `next(g)` → 从函数开头或上次暂停处继续执行 2. 遇到 `yield` → 返回值，暂停 3. 下次再调 `next(g)` → 从暂停处继续 4. 没有 `yield` 了 → 抛 `StopIteration`

4. 场景

场景 1：处理大文件（流式读取）

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

# 不管文件多大，一次只读一行到内存
for line in read_large_file("100gb_log.txt"):
    if "ERROR" in line:
        print(line)
```

如果不这么做，`f.readlines()` 会把整个文件读进内存。

场景 2：无限序列

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
print([next(fib) for _ in range(10)])
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# 可以一直取，不会内存溢出
```

场景 3：管道式数据处理

```
def read_ints():
    for i in range(100):
        yield i

def even_only(nums):
    for n in nums:
        if n % 2 == 0:
            yield n

def multiply_by_10(nums):
    for n in nums:
        yield n * 10

# 数据管道 — 每个生成器只处理一步，不创建中间列表
result = multiply_by_10(even_only(read_ints()))
for r in result:
    print(r)
```

5. 替代方案对比

方案	内存	延迟计算	无限序列	消费方式
生成器	低（惰性）	是	支持	只能遍历一次
列表推导式	高（全部在内存）	否	不支持	可多次遍历
生成器表达式	低	是	支持	只能遍历一次

列表 vs 生成器：

```
# 列表 — 全部算完，占内存
squares_list = [x*x for x in range(100_000_000)] # 可能 OOM

# 生成器表达式 — 惰性，几乎不占内存
squares_gen = (x*x for x in range(100_000_000)) # 没问题

# 生成器函数 — 跟表达式等价，但可以更复杂
def squares():
    for x in range(100_000_000):
        yield x*x
```

6. 常见坑

坑 1：生成器只能遍历一次

```
gen = (x for x in range(5))
print(list(gen)) # [0, 1, 2, 3, 4]
print(list(gen)) # [] — 已经耗尽
```

```
# 如果以后还要用，生成列表
gen = [x for x in range(5)] # 列表
```

坑 2：生成器只在你要求时才计算

```
def dangerous():
    print("正在执行危险操作...")
    yield "结果"

g = dangerous() # 没有打印！函数还没执行
# 100 行代码后...
result = next(g) # 才真正执行
```

这既是优点（惰性）也是陷阱（你以为执行了，实际上没有）。

坑 3：send() 给 yield 传值（高级，了解即可）

```
def accumulator():
    total = 0
    while True:
        value = yield total # yield 表达式可以接收值
        total += value

acc = accumulator()
next(acc) # 必须先启动到第一个 yield
print(acc.send(10)) # 10 — send 给 yield 赋值并推进
print(acc.send(5)) # 15
```

7. 代码验证

```
def batch_reader(data, batch_size=3):
    """将数据分批处理"""
    batch = []
    for item in data:
        batch.append(item)
        if len(batch) == batch_size:
            yield batch
            batch = []
    if batch: # 最后不足一批的
        yield batch

data = [1, 2, 3, 4, 5, 6, 7]
for batch in batch_reader(data):
```

```

    print(batch)
# [1, 2, 3]
# [4, 5, 6]
# [7]

```

五、上下文管理器（Context Manager）

1. 是什么

上下文管理器是 Python 中定义“进入 → 操作 → 退出”三段式流程的协议。通过 `with` 语句使用，保证“退出”操作一定会执行。

2. 解决了什么问题

大量资源操作有固定模式：申请资源 → 使用 → 释放资源。问题是释放这一步最容易被遗忘或跳过（比如函数中间 `return`、发生异常）。

上下文管理器让 Python 语言本身来保证：不管你怎么退出 `with` 块，`__exit__` 都会跑。

3. 核心理论

`with` 语句的完整执行流程：

```

with EXPR as VAR:
    BLOCK

```

Python 处理为（伪代码）：

```

manager = EXPR          # 计算表达式
VAR = manager.__enter__() # 进入
try:
    BLOCK                # 执行代码块
finally:
    manager.__exit__(...) # 一定退出（不管有没有异常）

```

`__exit__` 的四个参数：- `exc_type` — 异常类型（无异常为 `None`） - `exc_val` — 异常实例 - `exc_tb` — `traceback` 对象 - 返回值：`False` = 不吞异常；`True` = 吞掉异常

4. 场景

场景 1：资源管理

```

# 文件
with open("data.txt") as f:
    data = f.read()
# 自动 f.close()，哪怕中间有异常

# 数据库连接
class DBConnection:
    def __enter__(self):
        self.conn = sqlite3.connect("db.sqlite")
        return self.conn
    def __exit__(self, *args):
        self.conn.close()

# 锁
import threading

```

```
lock = threading.Lock()
with lock:
    # 自动 acquire/release, 不会死锁
    shared_data += 1
```

场景 2：临时状态切换

```
import sys
from io import StringIO

class RedirectStdout:
    def __enter__(self):
        self.old = sys.stdout
        sys.stdout = StringIO()
        return sys.stdout
    def __exit__(self, *args):
        captured = sys.stdout.getvalue()
        sys.stdout = self.old
        if captured:
            print(f"捕获到的输出: {captured.strip()}")
        return False

with RedirectStdout() as buf:
    print("这段不会显示在终端")
    print("会被捕获到 buf 里")
# 打印: 捕获到的输出: 这段不会显示在终端\n会被捕获到 buf 里
```

场景 3：事务管理

```
class Transaction:
    def __init__(self, conn):
        self.conn = conn
    def __enter__(self):
        self.conn.execute("BEGIN")
        return self
    def __exit__(self, exc_type, *args):
        if exc_type is None:
            self.conn.execute("COMMIT")
        else:
            self.conn.execute("ROLLBACK")
        return False # 不吞异常
```

5. 替代方案对比

方案	清理保证	可复用	代码噪声
with + 上下文管理器	语言保证	一次定义多处用	低
try/finally	靠人写	每次重写	高
try/except/finally	同上	同上	更高

6. 常见坑

坑 1：__exit__ 返回 True 吞异常

```
class SilenceEverything:
    def __exit__(self, *args):
        return True

with SilenceEverything():
    1 / 0 # 不会有 ZeroDivisionError!
print("这行能跑到") # 会被打印, 异常被吞了
```

一般情况不要返回 **True**。默认 **False** 是正确的选择。

坑 2：@contextmanager 一定要用 try/finally

```
from contextlib import contextmanager

@contextmanager
def good_timer():
    start = time.time()
    try:
        yield
    finally:
        print(f"耗时 {time.time() - start:.3f}s")

# 如果 with 块里抛异常，finally 保证取结束时间
```

坑 3：不是所有“两个操作”都适合做成上下文管理器

上下文管理器适合“做什么之前/之后”的配对操作。纯粹的两个独立步骤不适合。

7. 简写版：@contextmanager

```
from contextlib import contextmanager

@contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield          # yield 之前 = __enter__, yield 之后 = __exit__
    finally:
        elapsed = time.perf_counter() - start
        print(f"耗时 {elapsed:.3f}s")

with timer():
    sum(range(10_000_000))
```

8. 代码验证

```
import time

class Timer:
    """计时器上下文管理器"""
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.perf_counter() - self.start
        print(f"耗时 {elapsed:.3f}s")
        return False  # 不吞异常

with Timer():
    sum(range(10_000_000))
```

六、五个概念之间的关系

这五个概念不是孤立的，它们可以组合使用：

迭代器 ↔ **生成器**：生成器是迭代器的便捷实现。所有生成器都是迭代器。

装饰器 + 上下文管理器

```
def timed_context(cls):
    """装饰一个上下文管理器类，自动计时"""
    import functools
    original_exit = cls.__exit__

    @functools.wraps(original_exit)
    def timed_exit(self, *args):
        elapsed = time.perf_counter() - self._start
        print(f"{cls.__name__} 耗时 {elapsed:.3f}s")
        return original_exit(self, *args)

    cls.__exit__ = timed_exit
    return cls

@timed_context
class FileHandler:
    def __enter__(self):
        self._start = time.perf_counter()
        return self
    def __exit__(self, *args):
        print("清理资源")
        return False
```

生成器 + 上下文管理器 (@contextmanager)

```
from contextlib import contextmanager

@contextmanager
def open_file(path, mode='r'):
    """结合了生成器和上下文管理器"""
    f = open(path, mode)
    try:
        yield f
    finally:
        f.close()

with open_file("test.txt") as f:
    data = f.read()
```

装饰器 + 生成器

```
def log_yields(func):
    """日志记录生成器每次 yield 的值"""
    import functools
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        gen = func(*args, **kwargs)
        for value in gen:
            print(f"yield: {value}")
            yield value
        return wrapper

    return wrapper

@log_yields
def count_up(n):
    for i in range(n):
        yield i

for x in count_up(3):
    pass
# yield: 0
# yield: 1
# yield: 2
```


总结

概念	一句话	跟其他概念的关系
迭代器	逐个返回元素， __iter__ + __next__	生成器是它的简便实现
闭包	函数记住外部变量	装饰器的基础
装饰器	不修改代码给函数加功能	基于闭包实现
生成器	可暂停和恢复的函数 (yield)	是迭代器，也能配合上下文管理器 (@contextmanager)
上下文管理器	保证进入和退出配对执行	可以用生成器简写 (@contextmanager)

这五个概念共同体现了 Python 的一个核心理念：把重复的模式抽象出来，让语言替你做正确的事。

Day 3 — 标准库兵器谱（完整版）

一、collections — 比内置容器更强大的工具

1. 是什么

collections 是 Python 标准库中提供“增强版容器”的模块。它填补了内置类型（list、dict、set、tuple）在实际工程中的常见缺口。

常用的类：

类名	一句话	替代谁
defaultdict	访问不存在的 key 时不报错，自动创建	dict
Counter	计数+排序一步到位	手动 dict 计数
deque	两端操作都是 O(1) 的列表	list（当队列用）
namedtuple	有字段名的元组	普通元组、简单类
ChainMap	多层字典合并查找	多个字典
OrderedDict	记住插入顺序（3.7 后 dict 也记住了）	普通 dict

2. 解决了什么问题

问题 1：字典分组时要重复检查 key 存在

```
# 不用 defaultdict 的写法
word_counts = {}
for word in words:
    if word not in word_counts: # □ 每次都检查
        word_counts[word] = 0
    word_counts[word] += 1

# 更糟的是把值也做成列表
groups = {}
```

```
for item in items:
    key = item["category"]
    if key not in groups:
        groups[key] = []
    groups[key].append(item)
```

这段代码的“先检查再写”模式在 Python 中太常见了，以至于有必要用一个专门的类来消除样板代码。

问题 2：计数场景太频繁

按种类统计、频次分析、排行榜……这些需求写手动 `dict.get(key, 0) + 1` 太重复。

问题 3：list 当队列用性能差

`list.pop(0)` 和 `list.insert(0, x)` 都是 $O(n)$ 操作——每次都会移动所有元素。

问题 4：轻量数据结构要写类

只是想表示一个 (x, y) 点，不想写一个完整的 class——但又想用 `.x` 而不是 `[0]`。

3. 核心理论

defaultdict

```
from collections import defaultdict

# 核心思想：给 dict 一个“工厂函数”，访问不存在的 key 时自动调用它创建默认值
dd = defaultdict(int)
# 等价于：
# class MyDict(dict):
#     def __missing__(self, key):
#         self[key] = int()
#         return self[key]

dd["a"] += 1 # "a" 不存在 → int() → 0 → +=1 → 1
print(dd)   # defaultdict(<class 'int'>, {'a': 1})
```

工厂函数必须是可调用的无参函数或 None：

```
# 常见工厂
defaultdict(int)      # → 0
defaultdict(float)    # → 0.0
defaultdict(str)      # → ""
defaultdict(list)     # → []
defaultdict(set)       # → set()
defaultdict(bool)     # → False
defaultdict(lambda: "N/A") # 自定义

# None 的特殊行为
dd = defaultdict(None)
# print(dd["x"]) # ✗ TypeError: 'NoneType' object is not callable
```

Counter

Counter 是 dict 的子类，所以可以用所有 dict 方法：

```
from collections import Counter

c = Counter("abracadabra")
print(c) # Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

```
# dict 方法全能用
for key in c:
    print(key, c[key])

print(c.keys())    # dict_keys(['a', 'b', 'r', 'c', 'd'])
print(c.values())  # dict_values([5, 2, 2, 1, 1])
```

Counter 的独有方法：

```
# most_common — 频率排序
c.most_common()      # [('a',5), ('b',2), ('r',2), ('c',1), ('d',1)]
c.most_common(2)     # [('a',5), ('b',2)]
c.most_common()[:3:-1] # 最少的两个

# elements — 按计数展开
list(c.elements())   # ['a','a','a','a','a','b','b','r','r','c','d']

# subtract — 批量减法
c.subtract("car")
print(c) # Counter({'a': 4, 'b': 2, 'r': 1, 'c': 0, 'd': 1})

# total (3.10+) — 元素总数
c.total() # 11 (5+2+2+1+1)
```

Counter 的数学运算：

```
c1 = Counter(a=3, b=1, c=0)
c2 = Counter(a=1, b=2, d=3)

c1 + c2 # Counter({'a': 4, 'b': 3, 'd': 3}) 相加（只保留正数）
c1 - c2 # Counter({'a': 2})                相减（只保留正数）
c1 & c2 # Counter({'a': 1, 'b': 1})         交集（取最小值）
c1 | c2 # Counter({'a': 3, 'b': 2, 'd': 3}) 并集（取最大值）
+c1     # Counter({'a': 3, 'b': 1})         移除零和负数
-c1     # Counter()                        只保留负数转正（这里没有）
```

deque — 双端队列

deque 内部用分段数组（blocks）实现，两端操作都是 O(1)：

```
from collections import deque

dq = deque(maxlen=3) # 固定大小，超出的自动丢弃
dq.append(1)
dq.append(2)
dq.append(3)
dq.append(4) # 1 自动从左侧弹出
print(dq)    # deque([2, 3, 4], maxlen=3)

# 两端操作 O(1)
dq.appendleft(0) # deque([0, 2, 3])
dq.pop()         # 3, deque([0, 2])
dq.popleft()     # 0, deque([2])
dq.extend([4, 5]) # deque([2, 4, 5])
dq.extendleft([1]) # deque([1, 2, 4, 5]) 注意 extendleft 反转输入

# 旋转
dq.rotate(1) # deque([5, 1, 2, 4]) 向右移
dq.rotate(-1) # deque([1, 2, 4, 5]) 向左移
```

namedtuple

```

from collections import namedtuple

# 创建（类名， 字段列表）
Point = namedtuple("Point", ["x", "y"])

# 用法
p = Point(3, 4)
p.x          # 3 — 属性访问
p[0]         # 3 — 索引访问
x, y = p     # 解包

# 关键特性：不可变
# p.x = 5    # ✕ AttributeError

# 修改需要创建新对象
p2 = p._replace(x=10) # Point(x=10, y=4)
p3 = Point(x=5, y=p.y) # 手动构造

# 转字典
p._asdict() # {'x': 3, 'y': 4}

# 字段信息
p._fields   # ('x', 'y')

```

ChainMap

ChainMap 不合并字典——它在查找时按顺序搜索各个字典：

```

from collections import ChainMap

defaults = {"theme": "light", "lang": "en", "debug": False}
user = {"theme": "dark", "lang": "zh"} # 用户覆盖
runtime = {"debug": True} # 运行时覆盖

config = ChainMap(runtime, user, defaults)
# 查找顺序：runtime → user → defaults

config["theme"] # "dark" (来自 user)
config["debug"] # True (来自 runtime)
config["lang"] # "zh" (来自 user)
config.get("font_size", "12px") # "12px"

# 写入永远影响第一个映射
config["timeout"] = 30 # 写入 runtime
config["theme"] = "light" # 覆盖 runtime 的 theme
del config["theme"] # 从 runtime 删除，但 user 还在
config["theme"] # "dark" (来自 user)

# new_child — 增加一层
config2 = config.new_child({"debug": False})
# 现在四层：新字典 → runtime → user → defaults

```

4. 场景

场景 1：树形结构（defaultdict 嵌套）

```

from collections import defaultdict
import json

# 用 defaultdict 自动构建嵌套字典
def tree():
    return defaultdict(tree)

```

```

categories = tree()
categories["fruit"]["red"]["apple"] = 3
categories["fruit"]["red"]["strawberry"] = 5
categories["fruit"]["yellow"]["banana"] = 2
categories["animal"]["mammal"]["dog"] = 1

print(json.dumps(categories, indent=2))
# {
#   "fruit": {
#     "red": {"apple": 3, "strawberry": 5},
#     "yellow": {"banana": 2}
#   },
#   "animal": {
#     "mammal": {"dog": 1}
#   }
# }

```

这种模式叫“递归 defaultdict”——访问任意深度的 key 都会自动创建路径上的所有父节点。

场景 2：滑动窗口（deque）

```

from collections import deque

class MovingAverage:
    def __init__(self, size: int):
        self.window = deque(maxlen=size)
        self.sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.window.maxlen:
            self.sum -= self.window[0] # 减去即将被弹出的
        self.window.append(val)
        self.sum += val
        return self.sum / len(self.window)

ma = MovingAverage(3)
print(ma.next(1)) # 1.0
print(ma.next(10)) # 5.5
print(ma.next(3)) # 4.666...
print(ma.next(5)) # 6.0 (窗口: [10, 3, 5])

```

场景 3：配置文件层级（ChainMap）

```

from collections import ChainMap

# 模拟 bash 的环境变量查找
env = ChainMap()

# 加载不同层级的配置
with open("envs/prod.env") as f:
    env = env.new_child(parse_env_file(f))

with open("secrets.env") as f:
    env = env.new_child(parse_env_file(f))

# 运行时环境变量优先级最高
env = env.new_child(os.environ)

print(env["DATABASE_URL"]) # 自动按优先级查找

```

5. 替代方案对比

场景	不用 collections	用 collections	结论
分组	dict.setdefault / 反复 if key in dict	defaultdict(list)	defaultdict 更短
计数	dict.get(k, 0) + 1 / try-except	Counter	Counter 还有排序
双端操作	list.pop(0)	deque.popleft()	O(1) vs O(n)
数据类	写 class + __init__	namedtuple / dataclass	namedtuple 不可变
多层配置	手动合并 dict	ChainMap	ChainMap 不拷贝

6. 常见坑

```
# 坑 1: defaultdict 的默认值会隐式创建 key
dd = defaultdict(list)
print("a" in dd)      # False — 没访问就不创建
print(dd["a"])        # [] — 创建了
print("a" in dd)      # True — 隐式创建了

# 坑 2: defaultdict 的工厂函数只调用一次
dd = defaultdict(dict)
dd["a"]["x"] = 1      # ✓ 可以, 因为 dict() 返回新 dict
# dd["a"]["y"]["z"] = 2 # ✗ TypeError — "y"不存在

# 坑 3: Counter 访问不存在的 key 返回 0 (不会报错)
c = Counter()
print(c["nonexistent"]) # 0 — 可能隐藏 bug

# 坑 4: deque maxlen 是无边界的上限, 不是固定长度
dq = deque(maxlen=3)
dq.append(1)
dq.append(2)
print(len(dq))        # 2, 还没满
dq.append(3)
print(len(dq))        # 3, 满了
dq.append(4)          # 1 被自动弹出, len 还是 3

# 坑 5: namedtuple 不可变—忘了这点会踩坑
Point = namedtuple("Point", ["x", "y"])
p = Point(1, 2)
# p.x = 3 # ✗ AttributeError — 要明白这是 tuple
```

7. 代码验证

```
# 验证 ChainMap 不拷贝
from collections import ChainMap

a = {"x": 1}
b = {"y": 2}
cm = ChainMap(a, b)

cm["x"] = 100        # 修改 ChainMap
print(a["x"])        # 100 — 原字典也被改了!

cm.new_child({"z": 3})
print(len(cm.maps))  # 还是 2 — new_child 返回新 ChainMap, 不修改原来的

# 验证 deque 两端复杂度
from collections import deque
```

```
import time
```

```
n = 1000000
```

```
# list 左端插入
```

```
lst = []
```

```
t0 = time.perf_counter()
```

```
for i in range(n):
```

```
    lst.insert(0, i)
```

```
print(f"list.insert(0): {time.perf_counter() - t0:.3f}s")
```

```
# deque 左端插入
```

```
dq = deque()
```

```
t0 = time.perf_counter()
```

```
for i in range(n):
```

```
    dq.appendleft(i)
```

```
print(f"deque.appendleft: {time.perf_counter() - t0:.3f}s")
```

```
# 实测差距: list 是  $O(n^2)$ , deque 是  $O(n)$ 
```

二、itertools — 迭代器瑞士军刀

1. 是什么

itertools 是 Python 标准库中最被低估的模块之一。它提供了用于构建迭代器管道的工具函数——每个函数都返回一个迭代器（惰性求值），可以组合成数据处理链。

核心思想：不要一次性加载全部数据，用迭代器逐个处理。

2. 解决了什么问题

- 处理大数据时不想全部加载到内存
- 常见的遍历模式（无限序列、组合、分组、变换、切片）需要重复实现
- 嵌套循环的代码又慢又丑

3. 核心理论

所有 itertools 函数都返回**迭代器**——不立即计算，只在被消费时才逐个产生值。这意味着你可以构建无限长的链而不消耗内存。

分类如下：

类别	函数	作用
无限	count	无限计数
无限	cycle	无限循环
无限	repeat	无限重复
组合	product	笛卡尔积
组合	permutations	排列
组合	combinations	组合
组合	combinations_with_replacement	可重复组合
变换	chain	串接迭代器
变换	compress	布尔过滤
变换	dropwhile/takewhile	条件跳过/取

类别	函数	作用
变换	filterfalse	反向过滤
变换	starmap	展开参数映射
变换	accumulate	累积运算
分组	groupby	分组（必须预排序）
切片	islice	迭代器切片
新式	pairwise (3.10+)	相邻对
新式	batched (3.12+)	分批处理

无限迭代器

```

from itertools import count, cycle, repeat

# count(start=0, step=1) — 无限计数
for i in count(start=10, step=2):
    if i > 20:
        break
    print(i) # 10, 12, 14, 16, 18, 20

# 常见使用：给数据加序号
for idx, item in zip(count(1), data):
    print(idx, item)

# cycle(iterable) — 无限循环
colors = cycle(["red", "green", "blue"])
for _ in range(5):
    print(next(colors)) # red, green, blue, red, green

# 实战：轮询多个端点
endpoints = cycle(["server1", "server2", "server3"])
for _ in range(10):
    server = next(endpoints)
    check_health(server)

# repeat(object, times=None) — 无限或有限重复
for x in repeat("ping", 3):
    print(x) # ping, ping, ping

# 常用：map 的恒定参数
list(map(pow, range(5), repeat(2)))
# [0, 1, 4, 9, 16] — 相当于 [x**2 for x in range(5)]

```

组合迭代器

```

from itertools import product, permutations, combinations

# product(*iterables, repeat=1) — 笛卡尔积
list(product("AB", [1, 2]))
# [('A', 1), ('A', 2), ('B', 1), ('B', 2)]

# repeat 参数用于自身乘积
list(product("AB", repeat=2))
# [('A', 'A'), ('A', 'B'), ('B', 'A'), ('B', 'B')]

# 替代嵌套循环
# 不用 product:
for x in range(3):
    for y in range(3):
        process(x, y)

```



```
# 用 product:
for x, y in product(range(3), range(3)):
    process(x, y)
# 嵌套越深, product 优势越大

# permutations(iterable, r=None) - 排列
list(permutations("ABC", 2))
# [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
# 注意顺序有关, ('A', 'B') ≠ ('B', 'A')

# combinations(iterable, r) - 组合 (不放回)
list(combinations("ABC", 2))
# [('A', 'B'), ('A', 'C'), ('B', 'C')]

# combinations_with_replacement - 组合 (可重复)
list(combinations_with_replacement("ABC", 2))
# [('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'B'), ('B', 'C'), ('C', 'C')]
```

变换与过滤

```
from itertools import chain, compress, dropwhile, takewhile, filterfalse, starmap, accumulate

# chain - 串接多个可迭代对象
list(chain([1, 2], [3, 4], "ab"))
# [1, 2, 3, 4, 'a', 'b']

# chain.from_iterable - 展开嵌套
list(chain.from_iterable([[1, 2], [3, 4], [5]]))
# [1, 2, 3, 4, 5]

# compress - 按 mask 过滤 (比列表推导快)
data = "ABCDEF"
selectors = [1, 0, 1, 0, 1, 0]
list(compress(data, selectors))
# ['A', 'C', 'E']

# dropwhile - 跳过开头满足条件的
list(dropwhile(lambda x: x < 5, [1, 4, 6, 3, 7]))
# [6, 3, 7] - 遇到第一个不满足的之后, 后面的都不跳过

# takewhile - 取开头满足条件的 (遇到第一个不满足的停止)
list(takewhile(lambda x: x < 5, [1, 4, 6, 3, 7]))
# [1, 4]

# filterfalse - 过滤器: 取不满足条件的
list(filterfalse(lambda x: x % 2, range(10)))
# [0, 2, 4, 6, 8]

# starmap - 解包参数映射
from operator import pow
list(starmap(pow, [(2, 3), (3, 2), (4, 2)]))
# [8, 9, 16] - 相当于 pow(2,3), pow(3,2), pow(4,2)

# accumulate - 累积 (默认是累加)
list(accumulate([1, 2, 3, 4, 5]))
# [1, 3, 6, 10, 15]

from operator import mul
list(accumulate([1, 2, 3, 4, 5], mul))
# [1, 2, 6, 24, 120]
```

分组与切片

```

from itertools import groupby, islice, pairwise, batched

# groupby — 相邻分组（必须先排序，否则不准！）
data = [
    ("fruit", "apple"), ("fruit", "banana"),
    ("animal", "dog"), ("fruit", "cherry"),
]
# △ 没排序的结果：
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))
# fruit [('fruit', 'apple'), ('fruit', 'banana')]
# animal [('animal', 'dog')]
# fruit [('fruit', 'cherry')] ← 又出现一次 fruit！

# ✓ 正确用法：先排序
sorted_data = sorted(data, key=lambda x: x[0])
for key, group in groupby(sorted_data, key=lambda x: x[0]):
    print(key, len(list(group))) # fruit: 3, animal: 1

# islice — 迭代器切片
for line in islice(open("large.log"), 5): # 前 5 行
    print(line.strip())

# 也支持 start, stop, step
list(islice(range(10), 2, 8, 2)) # [2, 4, 6]

# pairwise (3.10+) — 相邻对
list(pairwise([1, 2, 3, 4]))
# [(1, 2), (2, 3), (3, 4)]

# 实战：检测相邻差值
diffs = [b - a for a, b in pairwise(data)]

# batched (3.12+) — 分批
list(batched(range(10), 3))
# [(0, 1, 2), (3, 4, 5), (6, 7, 8), (9,)]

```

4. 场景

场景 1：逐块读取大文件（islice + batched）

```

from itertools import islice, batched

def read_in_chunks(filepath, chunk_size=1000):
    """以行组方式读大文件"""
    with open(filepath) as f:
        for chunk in batched(f, chunk_size):
            yield [line.strip() for line in chunk]

# 读前 100 个块
for chunk in islice(read_in_chunks("huge.log"), 100):
    process_chunk(chunk)

```

场景 2：带索引的分页（count）

```

from itertools import count, islice

class Paginator:
    def __init__(self, items, page_size=10):
        self._items = items
        self.page_size = page_size

```

```

def get_page(self, page_num):
    start = (page_num - 1) * self.page_size
    return list(
        islice(self._items, start, start + self.page_size)
    )

def all_pages(self):
    for i in count(1):
        page = self.get_page(i)
        if not page:
            break
        yield i, page

```

场景 3：排列组合测试用例（product）

```

from itertools import product

# 生成所有测试参数组合
params = {
    "db": ["sqlite", "postgres", "mysql"],
    "cache": ["redis", "memcached", "none"],
    "debug": [True, False],
}

test_cases = list(product(*params.values()))
print(len(test_cases)) # 3 × 3 × 2 = 18

for db, cache, debug in test_cases:
    run_test(db=db, cache=cache, debug=debug)

# 用 dict 理解：
for combo in product(*params.values()):
    case = dict(zip(params.keys(), combo))
    run_test(**case)

```

5. 替代方案对比

场景	itertools	替代方案	选哪个
无限序列	count	while 循环	itertools 更干净
嵌套循环	product	多重 for	嵌套 ≤2 层时随意， ≥3 层用 product
组合/排列	permutations/combinations	手动递归	永远用 itertools
串接迭代器	chain	list1 + list2	大数据用 chain(惰性)， 小数据随意
分组	groupby	defaultdict	需要排序/有序时用 groupby，否则 defaultdict
切片	islice	list slicing	大数据用 islice

6. 常见坑

```

# 坑 1: groupby 必须先排序！
# 见上面的例子，不排序会得到多个相同 key 的组

# 坑 2: itertools 返回迭代器——一次消费就没了
from itertools import product
p = product("AB", [1, 2])
print(list(p)) # [('A', 1), ('A', 2), ('B', 1), ('B', 2)]

```

```

print(list(p)) # [] - 耗尽了

# 坑 3: islice 的负索引和步进是贪婪的
# islice 不支持负索引 (不像列表)
# list(islice(range(10), -3)) # ✗ 无效
# list(islice(range(10), -3, None)) # ✗ 无效

# 坑 4: chain.from_iterable 只展平一层
nested = [[1, 2], [3, [4, 5]]]
list(chain.from_iterable(nested))
# [1, 2, 3, [4, 5]] - [4, 5] 没展开

```

7. 代码验证

```

# 验证: product 的性能优势
from itertools import product
import time

# 4 层嵌套循环
def nested_loops():
    result = []
    for a in range(10):
        for b in range(10):
            for c in range(10):
                for d in range(10):
                    result.append((a, b, c, d))
    return result

def product_loop():
    return list(product(range(10), repeat=4))

t0 = time.perf_counter()
nested_loops()
t1 = time.perf_counter()
product_loop()
t2 = time.perf_counter()

print(f"nested: {t1-t0:.4f}s")
print(f"product: {t2-t1:.4f}s")
# product 通常略快, 但主要优势是代码简洁

# 验证: islice 的惰性求值
from itertools import islice

def expensive_generator():
    for i in range(10):
        print(f" generating {i}")
        yield i

print("Before islice:")
gen = expensive_generator()
sliced = islice(gen, 2)

print("After islice:")
print(list(sliced)) # 到这里才按需生成

# 输出顺序:
# Before islice:
# After islice:
#   generating 0
#   generating 1
# [0, 1]

```

三、functools — 函数工具箱

1. 是什么

functools 是专门对函数进行“元操作”的模块——它接受函数作为输入，返回（通常）另一个函数。

2. 解决了什么问题

- 想固定函数的某些参数（partial）
- 想自动缓存函数结果提高性能（lru_cache / cache）
- 想根据参数类型选择不同实现（singledispatch）
- 想累计运算数据（reduce，虽然现在有更现代的替代）
- 想保留装饰后函数的元数据（wraps）

3. 核心理论

partial — 部分应用（固定参数）

```
from functools import partial

def power(base, exp):
    return base ** exp

# 固定 exp 参数
square = partial(power, exp=2)
cube = partial(power, exp=3)

print(square(5))    # 25
print(cube(5))      # 125
print(square(10))   # 100

# 实战：简化函数调用
def request(method, url, headers=None, timeout=30):
    ...

get = partial(request, "GET")
post = partial(request, "POST")

get("/api/users", timeout=5)    # 不用传 method
post("/api/users", {"name": "Leo"})

# partial 的工作原理（简化版）
def my_partial(func, /, *args, **keywords):
    def wrapper(*fargs, **fkeywords):
        new_keywords = {**keywords, **fkeywords}
        return func(*args, *fargs, **new_keywords)
    return wrapper
```

partial 在 Python 中的使用非常频繁——几乎所有带 callback 的库（Tkinter、asyncio、multiprocessing）都有它的身影。

lru_cache / cache — 自动记忆化

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    """斐波那契数列（带缓存）"""
```

```

    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

# 第一次调用：正常计算
# 后续调用：从缓存读

fibonacci(40) # 瞬间完成
fibonacci.cache_info()
# CacheInfo(hits=38, misses=41, maxsize=128, currsize=41)

@lru_cache(maxsize=None) # 无限制缓存
def expensive_io(path):
    ...

# Python 3.9+ 的简写
from functools import cache
@cache # 等价于 @lru_cache(maxsize=None)
def fib(n):
    ...

```

性能对比：

```

import time

def fib_raw(n):
    return n if n < 2 else fib_raw(n-1) + fib_raw(n-2)

@cache
def fib_cached(n):
    return n if n < 2 else fib_cached(n-1) + fib_cached(n-2)

t0 = time.perf_counter()
fib_raw(35)
print(f"raw: {time.perf_counter()-t0:.3f}s")

t0 = time.perf_counter()
fib_cached(35)
print(f"cached: {time.perf_counter()-t0:.3f}s")
# raw 可能数秒，cached 瞬间

```

singledispatch — 单分派泛型

```

from functools import singledispatch

@singledispatch
def serialize(obj):
    """默认实现"""
    raise TypeError(f"Unsupported type: {type(obj)}")

@serialize.register
def _(obj: int) -> str:
    return str(obj)

@serialize.register
def _(obj: float) -> str:
    return f"{obj:.2f}"

@serialize.register
def _(obj: str) -> str:
    return f'"{obj}"'

@serialize.register
def _(obj: list) -> str:

```

```

    items = ", ".join(serialize(x) for x in obj)
    return f"[{items}]"

@serialize.register
def _(obj: dict) -> str:
    pairs = ", ".join(f"{k}: {serialize(v)}" for k, v in obj.items())
    return f"{{{pairs}}}"

print(serialize(42))          # "42"
print(serialize(3.14159))     # "3.14"
print(serialize("hello"))     # "'hello'"
print(serialize([1, 2, 3]))   # "[1, 2, 3]"
print(serialize({"a": 1}))    # "{a: 1}"

# 自定义类型一样支持
@serialize.register
def _(obj: datetime) -> str:
    return obj.isoformat()

# 注册 None
@serialize.register(type(None))
def _(obj) -> str:
    return "null"

```

什么时候用 **singledispatch** 而不是 **if-else**: - 类型分支很多 (5+) - 需要第三方扩展注册新类型

wraps — 保留装饰器元数据

```

from functools import wraps

def my_decorator(f):
    @wraps(f) # 没有这一行, add.__name__ 会变成 "wrapper"
    def wrapper(*args, **kwargs):
        print(f"Calling {f.__name__}")
        return f(*args, **kwargs)
    return wrapper

@my_decorator
def add(a, b):
    """Add two numbers"""
    return a + b

print(add.__name__) # "add" (有 @wraps)
print(add.__doc__) # "Add two numbers"

```

4. 场景

场景 1: API 请求重试 (partial + decorator)

```

from functools import partial

def retry(func, max_retries=3):
    for attempt in range(max_retries):
        try:
            return func()
        except Exception:
            if attempt == max_retries - 1:
                raise

def fetch_data(url, params=None):
    ...

```

```
# 不修改原函数，创建特定版本
fetch_with_retry = partial(retry, max_retries=5)
safe_fetch = fetch_with_retry(partial(fetch_data, "/api/data"))
```

场景 2：缓存昂贵计算（lru_cache）

```
from functools import lru_cache
import requests

@lru_cache(maxsize=32, ttl=60) # ttl 需要 3.12+, 或自己实现
def get_weather(city: str) -> dict:
    """缓存在 60 秒内相同城市的天气查询"""
    resp = requests.get(f"https://api.weather.com/{city}")
    return resp.json()

# 也可以在运行时查看缓存命中情况
stats = get_weather.cache_info()
print(f"hits={stats.hits}, misses={stats.misses}")

# 手动清空
get_weather.cache_clear()
```

场景 3：序列化不同类型（singledispatch）

```
from functools import singledispatch
from datetime import datetime
from pathlib import Path

@singledispatch
def to_json(obj):
    return str(obj)

@to_json.register
def _(obj: datetime) -> str:
    return obj.isoformat()

@to_json.register
def _(obj: Path) -> str:
    return str(obj)

@to_json.register
def _(obj: Exception) -> str:
    return f"{type(obj).__name__}: {obj}"

# 还能按抽象类型注册
import numbers
@to_json.register
def _(obj: numbers.Real) -> str:
    return f"{obj:.6f}"
```

5. 替代方案对比

场景	functools	替代方案	选哪个
固定参数	partial	lambda	代码复用多时用 partial
记忆化	cache/lru_cache	手动 dict 缓存	纯函数用 cache，有 大小限制用 lru_cache
类型分派	singledispatch	if/elif/elif/...	类型少用 if，注册模 式用 singledispatch

场景	functools	替代方案	选哪个
装饰器元数据	wraps	手动复制 <code>__name__</code> / <code>__doc__</code>	永远用 wraps
归约	reduce	for 循环 / sum / 列表推导	3 行以上用 for, 否则 reduce

6. 常见坑

```
# 坑 1: partial 固定位置参数要小心
def f(a, b, c):
    pass

p = partial(f, 1, 2)
p(3) # ✓ f(1, 2, 3)

p = partial(f, 1)
p(2, 3) # ✓ f(1, 2, 3)

# 但混用位置和关键字要注意
p = partial(f, 1, c=3)
# p(2, c=4) # ✗ TypeError: got multiple values for 'c'

# 坑 2: lru_cache 的参数必须可哈希
@lru_cache
def process_list(lst):
    # 参数 lst 是 list (不可哈希) —— 会报错
    return sum(lst)
# ✗ TypeError: unhashable type: 'list'
# 改成元组即可: @lru_cache def process_tuple(t: tuple): ...

# 坑 3: singledispatch 是基于第一个参数的**类型注解**
# 没有类型注解时默认走主函数
@singledispatch
def f(obj):
    return "default"

@f.register
def _(obj): # ← 没有类型注解! 不会注册
    return "never called"

# 坑 4: reduce 代码可读性差
# ✗
from functools import reduce
result = reduce(lambda acc, x: acc | {x: x**2}, data, {})

# ✓
result = {x: x**2 for x in data}
```

7. 代码验证

```
# 验证 partial 与 lambda 的性能差异
from functools import partial
import time

def add(a, b):
    return a + b

# partial
add5_p = partial(add, 5)
# lambda
add5_l = lambda x: add(5, x)
```

```

# 功能一样
print(add5_p(3))    # 8
print(add5_l(3))    # 8

# partial 略快 (lambda 要多一层函数调用)
# 且 partial 保留了 func 信息
print(add5_p.func)  # <function add at ...>
print(add5_p.args)  # (5,)

# lambda 的 __name__ 是 "<lambda>"

# 验证 lru_cache 与手动缓存的等价性
from functools import lru_cache

call_count = 0

@lru_cache(maxsize=None)
def compute(n):
    global call_count
    call_count += 1
    return n * n

# 第一次：计算
compute(5)    # call_count = 1
compute(5)    # cache hit, call_count = 1
compute(10)   # call_count = 2
compute(5)    # cache hit, call_count = 2

print(compute.cache_info())
# CacheInfo(hits=2, misses=2, maxsize=None, currsize=2)

```

四、错误处理模式

1. 是什么

Python 有一套独特的错误处理哲学：**EAFP**（Easier to Ask Forgiveness than Permission，先试再求饶）。

这与其他语言的 **LBYL**（Look Before You Leap，先看再跳）形成鲜明对比。

2. 解决了什么问题

LBYL 的问题：- 检查和操作之间有**竞态条件**（检查通过了，操作时变了）- 检查的逻辑和操作的逻辑**重复**（都在描述同一个条件）- 检查越复杂，代码越长

```

# LBYL 的问题
if os.path.exists("config.json"):
    if os.access("config.json", os.R_OK):
        with open("config.json") as f:
            data = json.load(f)
    else:
        data = {}
else:
    data = {}
# 问题：检查完到打开之间文件可能被删/改权限

```

3. 核心理论

```

# EAFP – Python 推荐风格
try:

```

```

    with open("config.json") as f:
        data = json.load(f)
except FileNotFoundError:
    data = {}
except PermissionError:
    data = {}
except json.JSONDecodeError:
    data = {}

# 更简洁
try:
    with open("config.json") as f:
        data = json.load(f)
except (FileNotFoundError, PermissionError, json.JSONDecodeError):
    data = {}

```

异常链 (raise ... from) :

```

class AppError(Exception):
    """应用层异常"""
    pass

def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError as e:
        raise AppError(f"Config not found: {path}") from e
        # "from e" 保留了原始异常堆栈

try:
    load_config("missing.json")
except AppError as e:
    print(e)          # "Config not found: missing.json"
    print(e.__cause__) # FileNotFoundError
    import traceback
    traceback.print_exc()
    # 显示完整异常链

```

不要捕获所有异常:

```

# ✗ 反面教材
try:
    data = process_input(user_input)
except Exception: # 什么异常都吞了
    pass

# ✔ 明确指定
try:
    data = process_input(user_input)
except ValueError:
    data = default
except TypeError:
    data = default

```

4. 场景

场景 1: contextlib.suppress — 优雅忽略异常

```

from contextlib import suppress

# 不用 suppress:
try:

```

```

    os.remove("temp.txt")
except FileNotFoundError:
    pass

# 用 suppress:
with suppress(FileNotFoundError):
    os.remove("temp.txt")

# 多个异常
with suppress(FileNotFoundError, PermissionError):
    os.remove("protected.txt")

# 实战: 清理临时文件
with suppress(FileNotFoundError):
    os.unlink("/tmp/cache.dat")
with suppress(FileNotFoundError):
    shutil.rmtree("/tmp/build")

```

场景 2: 自定义异常体系

```

class APIError(Exception):
    """API 基础异常"""
    def __init__(self, message, status_code=None, response=None):
        super().__init__(message)
        self.status_code = status_code
        self.response = response

class NotFoundError(APIError):
    """资源不存在"""
    pass

class RateLimitError(APIError):
    """限流"""
    def __init__(self, message, retry_after=60):
        super().__init__(message, status_code=429)
        self.retry_after = retry_after

class AuthError(APIError):
    """认证失败"""
    pass

# 使用
try:
    response = api.get_user(42)
except NotFoundError:
    print("User not found, create one")
except RateLimitError as e:
    print(f"Rate limited, retry after {e.retry_after}s")
except AuthError:
    print("Check API key")

```

5. 常见坑

```

# 坑 1: except 的顺序会匹配
try:
    process()
except Exception:           # 先捕获所有异常
    print("caught")
except ValueError:          # 永远不会到这里
    print("never")

# ✓ 先具体后通用
try:

```

```

    process()
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
except Exception:          # 兜底
    print("Other error")

```

坑 2: try 块越大越好? 不一定

太大: 不知道哪一行出的错

```

try:
    data = load_file()
    parsed = parse(data)
    result = process(parsed)
    save(result)
except Exception:
    pass # 什么错了都不知道

```

太小: 代码充斥 try/except

```

try:
    data = load_file()
except Exception:
    ...
try:
    parsed = parse(data)
except Exception:
    ...

```

✓ 黄金法则: 一个 try 块做「一件事情」

```

try:
    data = load_file()
except FileNotFoundError:
    data = default

try:
    result = process(parse(data))
except ValueError:
    result = default

```

6. 代码验证

验证 EAFP 优于 LBYL 的场景

```
import time
```

准备数据: 10 个 key 中 3 个不存在

```

data = {i: i * 10 for i in range(7)}
keys = list(range(10))

```

LBYL

```

def lbyl():
    result = []
    for k in keys:
        if k in data:
            result.append(data[k] * 2)
    return result

```

EAFP

```

def eafp():
    result = []
    for k in keys:
        try:
            result.append(data[k] * 2)
        except KeyError:
            pass

```

```

    return result

print(lbyl())
print(eafp())
# 对存在的 key, EAFP 比 LBYL 快 (少一次查找)
# 对不存在的 key, EAFP 有异常开销
# 但大多数情况下, "不存在"是异常情况, EAFP 更合理

```

五、logging — 专业日志

1. 是什么

logging 是 Python 标准库的日志框架, 它提供了比 print() 专业得多的日志能力——包括级别过滤、输出目标、格式控制、日志轮转等。

2. 解决了什么问题

- print() 的输出和生产日志混在一起, 不知道哪个是调试信息
- 没有级别控制——debug 和 error 混在一起, 无法按严重程度过滤
- 输出目标单一——print() 只能到 stdout, 不能同时写文件和发邮件
- 没有格式化——纯文本, 缺少时间戳、行号、模块名
- 大型应用无法统一日志配置

3. 核心理论

日志级别 (由低到高)

级别	数值	什么时候用
DEBUG	10	详细信息, 仅开发调试
INFO	20	确认程序正常运行
WARNING	30	发生了不寻常但不是错误的事
ERROR	40	出错了但程序还能继续
CRITICAL	50	严重错误, 程序可能得退出

设置 level=logging.INFO 后, 低于此级别 (DEBUG) 的消息会被忽略。

基本用法

```

import logging

# 一次性配置 (建议在程序入口做)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    filename="app.log",      # 写到文件 (不写就输出到 stderr)
    filemode="a",           # 追加模式
)

# 获取 logger (用 __name__ 是标准惯例)
logger = logging.getLogger(__name__)

# 五种级别

```

```

logger.debug("Detailed info for debugging")
logger.info("Server started on port %d", 8080) # 支持 % 格式化
logger.warning("Disk space low: %.1f GB", 1.5)
logger.error("Failed to connect to database: %s", err)
logger.critical("System is out of memory!")

```

logger 的层级结构

```

# logger 按名字分层，用点号分隔
root = logging.getLogger() # 根 logger
app = logging.getLogger("app") # "app"
db = logging.getLogger("app.db") # "app.db" → app 的子 logger
api = logging.getLogger("app.api") # "app.api"

# 子 logger 默认继承父 logger 的设置
# 但可以独立设置 level、handler、filter

for name in ["app", "app.db", "app.api"]:
    logger = logging.getLogger(name)
    print(f"{name}: level={logger.level}, propagate={logger.propagate}")

```

Handler — 控制输出目标

```

import logging

logger = logging.getLogger("myapp")
logger.setLevel(logging.DEBUG)

# Handler 1: 控制台
console = logging.StreamHandler()
console.setLevel(logging.INFO)
console.setFormatter(logging.Formatter(
    "%(asctime)s %(message)s", datefmt="%H:%M:%S"
))

# Handler 2: 文件（每日轮转）
from logging.handlers import TimedRotatingFileHandler
file_handler = TimedRotatingFileHandler(
    "app.log",
    when="midnight", # 每天零点轮转
    backupCount=7, # 保留 7 天
    encoding="utf-8",
)
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(logging.Formatter(
    "%(asctime)s [%(levelname)s] %(name)s: %(lineno)d: %(message)s"
))

# Handler 3: 错误邮件（生产环境）
from logging.handlers import SMTPHandler
mail_handler = SMTPHandler(
    mailhost=("smtp.gmail.com", 587),
    fromaddr="bot@example.com",
    toaddrs=["admin@example.com"],
    subject="App Error Alert",
    credentials=("user", "pass"),
    secure=(),
)
mail_handler.setLevel(logging.ERROR)

# 注册所有 handler
logger.addHandler(console)

```

```
logger.addHandler(file_handler)
logger.addHandler(mail_handler)
```

4. 场景

场景 1：结构化 JSON 日志

```
import logging
import json

class JsonFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {
            "timestamp": self.formatTime(record),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        # 额外字段
        if hasattr(record, "user_id"):
            log_entry["user_id"] = record.user_id
        if hasattr(record, "request_id"):
            log_entry["request_id"] = record.request_id
        return json.dumps(log_entry, ensure_ascii=False)

# 使用
logger = logging.getLogger("api")
handler = logging.StreamHandler()
handler.setFormatter(JsonFormatter())
logger.addHandler(handler)

logger.info("User logged in", extra={"user_id": 42, "request_id": "abc-123"})
# {"timestamp": "2026-05-23 11:00:00", "level": "INFO", ...}
```

场景 2：异常日志 + 堆栈追踪

```
import logging

logger = logging.getLogger(__name__)

try:
    1 / 0
except ZeroDivisionError:
    # exc_info=True 自动记录完整堆栈
    logger.exception("Division failed") # 等价于 error + exc_info=True
    # 或:
    logger.error("Division failed", exc_info=True)
```

场景 3：性能日志

```
import logging
import time
from contextlib import contextmanager

logger = logging.getLogger("performance")

@contextmanager
def time_log(name):
    start = time.perf_counter()
    try:
        yield
```



```

finally:
    elapsed = time.perf_counter() - start
    if elapsed > 1.0:
        logger.warning("Slow operation: %s took %.3fs", name, elapsed)
    else:
        logger.debug("Operation: %s took %.3fs", name, elapsed)

# 使用
with time_log("database_query"):
    results = db.query("SELECT ...")

```

5. 替代方案对比

场景	logging	替代方案	结论
简单调试	print()	logger.debug	开发用 print，生产用 logging
结构化日志	logging + JSONFormatter	structlog / loguru	小项目用标准库够
性能敏感	控制日志级别	去掉日志	生产环境 INFO 级别
集中式日志	远程 handler	ELK / Loki / Datadog	配合标准格式输出

6. 常见坑

```

# 坑 1: 多次调用 basicConfig 无效
logging.basicConfig(level=logging.INFO) # ✔ 第一次
logging.basicConfig(level=logging.DEBUG) # ✘ 无效

# 解决方案: 提前配置, 或显式配置 logger
logger.setLevel(logging.DEBUG)

# 坑 2: 字符串格式化浪费性能
# ✘ 即使 level=WARNING, 字符串已拼接
logger.debug(f"Processing {len(data)} records")

# ✔ 用 % 格式化——logger 内部按 level 过滤后再拼接
logger.debug("Processing %d records", len(data))

# 坑 3: 异常日志不要手动传 traceback
try:
    1/0
except:
    # ✘ 多余
    logger.error(traceback.format_exc())
    # ✔ logging 自带
    logger.exception("Division error")

```

7. 代码验证

```

# 验证日志级别过滤
import logging

logging.basicConfig(level=logging.WARNING)
logger = logging.getLogger(__name__)

logger.debug("debug")      # ✘ 不输出
logger.info("info")       # ✘ 不输出
logger.warning("warn")    # ✔ 输出
logger.error("error")     # ✔ 输出
logger.critical("crit")   # ✔ 输出

```

六、正则表达式速查

1. 是什么

re 模块是 Python 的正则表达式引擎，用于字符串的模式匹配和替换。它兼容 Perl 风格的语法。

2. 解决了什么问题

- 验证格式（邮箱、手机号、URL）
- 提取特定模式的数据（从日志里提取 IP、日期）
- 批量替换（清理文本脱敏）
- 分割字符串（复杂分隔符）

3. 核心理论

基础匹配

```
import re

# match — 从开头匹配
re.match(r"\d+", "123abc")    # <Match object> — 匹配到 "123"
re.match(r"\d+", "abc123")    # None — 开头不是数字

# search — 搜索任意位置
re.search(r"\d+", "abc123def") # <Match object> — 匹配到 "123"

# fullmatch — 完全匹配
re.fullmatch(r"\d+", "123")    # <Match object>
re.fullmatch(r"\d+", "123a")   # None

# findall — 找所有匹配
re.findall(r"\d+", "a1b22c333") # ['1', '22', '333']

# finditer — 迭代匹配结果
for m in re.finditer(r"\d+", "a1b22c333"):
    print(m.group(), m.start(), m.end())
# 1 1 2
# 22 3 5
# 333 6 9
```

编译复用

```
# 多次使用相同模式时，编译后性能更好
pattern = re.compile(r"\b\w+@\w+\.\w+\b")

# 编译后的 pattern 有所有方法
pattern.search(text)
pattern.findall(text)
pattern.sub("[REDACTED]", text)

# 性能对比
import time

text = "test@example.com " * 1000

# 每次重新编译
t0 = time.perf_counter()
```

```
for _ in range(100):
    re.findall(r"\b\w+@\w+\.\w+\b", text)
print(f"uncompiled: {time.perf_counter() - t0:.3f}s")
```

编译后复用

```
t0 = time.perf_counter()
pattern = re.compile(r"\b\w+@\w+\.\w+\b")
for _ in range(100):
    pattern.findall(text)
print(f"compiled: {time.perf_counter() - t0:.3f}s")
```

分组

普通分组

```
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "Date: 2026-05-23")
m.group(0)    # "2026-05-23" — 完整匹配
m.group(1)    # "2026"
m.group(2)    # "05"
m.group(3)    # "23"
m.groups()    # ('2026', '05', '23')
```

命名分组 — 可读性更好

```
m = re.search(
    r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})",
    "Date: 2026-05-23"
)
m.group("year")    # "2026"
m.groupdict()      # {'year': '2026', 'month': '05', 'day': '23'}
```

不捕获分组

```
re.findall(r"(?:Mr|Ms|Dr)\.?\s+(\w+)", "Mr. Smith and Dr. Jones")
# ['Smith', 'Jones']
```

替换

简单替换

```
re.sub(r"\d+", "NUMBER", "abc123def456")
# "abcNUMBERdefNUMBER"
```

函数替换

```
def mask_credit_card(match):
    digits = match.group()
    return "****-****-****-" + digits[-4:]

re.sub(r"\d{4}-\d{4}-\d{4}-\d{4}", mask_credit_card,
    "Card: 1234-5678-9012-3456")
# "Card: ****-****-****-3456"
```

引用分组

```
re.sub(r"(\w+)@(\w+\.\w+)", r"\1 at \2",
    "Contact: user@example.com")
# "Contact: user at example.com"
```

常用模式速查

.	任意字符（除了换行）
\d	数字 [0-9]
\w	字母/数字/下划线 [a-zA-Z0-9_]
\s	空白符（空格、Tab、换行）
\b	单词边界
^	字符串开头
\$	字符串结尾

```

*          0 或多次
+          1 或多次
?          0 或 1 次
{n}        恰好 n 次
{n,}       至少 n 次
{n,m}      n 到 m 次
[abc]      a 或 b 或 c
[^abc]     不是 a/b/c
|          或
()         分组
(?:)       不捕获分组
(?P<name>) 命名分组

```

4. 场景

场景 1：解析日志

```

import re

LOG_PATTERN = re.compile(
    r"(?P<ip>\d+\.\d+\.\d+\.\d+)\s+"
    r"(?P<ident>\S+)\s+(?P<auth>\S+)\s+"
    r"\[(?P<datetime>[^\]]+)\]\s+"
    r'"(?P<method>\w+)\s+(?P<path>\S+)\s+(?P<proto>\S+)"\s+'
    r"(?P<status>\d{3})\s+(?P<size>\d+)"
)

def parse_access_log(line):
    m = LOG_PATTERN.match(line)
    if m:
        return m.groupdict()
    return None

# 使用
log_line = '192.168.1.1 - - [23/May/2026:10:30:45 +0000] "GET /api/users HTTP/1.1" 200 1234'
print(parse_access_log(log_line))
# {'ip': '192.168.1.1', 'datetime': '23/May/2026:10:30:45 +0000',
#  'method': 'GET', 'path': '/api/users', 'status': '200', ...}

```

场景 2：数据脱敏

```

import re

def mask_sensitive(text):
    """脱敏手机号和邮箱"""
    # 手机号中间四位
    text = re.sub(r"(1[3-9]\d)\d{4}(\d{4})", r"\1****\2", text)
    # 邮箱用户名部分
    text = re.sub(r"(\w)(\w+)(@)", lambda m: m.group(1) + "****" + m.group(3), text)
    return text

print(mask_sensitive("Contact: 13812345678, email@test.com"))
# "Contact: 138****5678, e****@test.com"

```

场景 3：模板替换

```

import re

def render_template(template, context):
    """简单的模板引擎: {{ name }} → value"""
    def replacer(match):

```

```

        key = match.group(1).strip()
        return str(context.get(key, match.group(0)))
    return re.sub(r"\{\{(\w+)\}\}", replacer, template)

template = "Hello {{ name }}, your order #{ order_id } is ready."
context = {"name": "Leo", "order_id": "12345"}
print(render_template(template, context))
# "Hello Leo, your order #12345 is ready."

```

5. 替代方案对比

场景	正则	替代方案	结论
固定字符串查找	re	str.find(), "abc" in text	用字符串方法
简单替换	re.sub	str.replace()	用 replace
复杂解析	命名分组	手写解析器	日志/配置用正则
HTML 解析	re	BeautifulSoup / lxml	用专用解析器
JSON 提取	re	json 模块	用 json

6. 常见坑

```

# 坑 1: 贪婪匹配
re.findall(r"<.*>", "<div><span>text</span></div>")
# ['<div><span>text</span></div>'] - 匹配了整个字符串

# ✓ 非贪婪: 加 ?
re.findall(r"<.*?>", "<div><span>text</span></div>")
# ['<div>', '<span>', '</span>', '</div>']

# 坑 2: 转义
# 匹配小数点
re.search(r"\d+\.\d+", "3.14") # ✓ \. 转义
re.search(r"\d+\d+", "3a14") # ✓ 但 . 没转义, 匹配了任意字符

# 在字符串中反斜杠也要转义
# 所以要用 r 原始字符串

# 坑 3: 回溯灾难
# 这个模式在处理长字符串时非常慢
pattern = re.compile(r"(a+)+b")
# pattern.match("a" * 30) # 可能卡住
# 解决方案: 避免嵌套量词

# 坑 4: ^ 和 $ 的多行模式
re.search(r"^abc", "123\nabc") # None

# ✓ 用 re.MULTILINE
re.search(r"^abc", "123\nabc", re.MULTILINE) # 匹配

# 坑 5: 反斜杠在 str 和 re 中的双重转义
# 匹配 \n 换行
re.search(r"\\n", "hello\\nworld") # ✓ raw string
re.search(r"\\\n", "hello\\nworld") # ✓ 但难读

```

7. 代码验证

```

# 验证编译 vs 不编译的性能
import re
import time

```

```
# 准备数据
text = "user@example.com " * 10000
pattern_str = r"\b[\w.+~]+@[~\w-]+\.[~\w.]+\b"
compiled = re.compile(pattern_str)

# 每次编译
t0 = time.perf_counter()
for _ in range(100):
    re.findall(pattern_str, text)
print(f"uncompiled: {time.perf_counter() - t0:.3f}s")

# 预编译
t0 = time.perf_counter()
for _ in range(100):
    compiled.findall(text)
print(f"compiled: {time.perf_counter() - t0:.3f}s")

# 差异在高频使用时会很大
```

总结

模块	核心价值	最常用的功能	一句话记法
collections	增强版容器	defaultdict、Counter、deque	不再手动写”if key in dict”
itertools	迭代器管道	product、chain、groupby、islice	用它替代嵌套循环
functools	函数元操作	partial、cache、wraps	固定参数、缓存结果
contextlib	简化异常处理	suppress	用 with 代替 try-except
logging	专业日志	getLogger、basicConfig	别用 print 了
re	模式匹配	compile、findall、sub	先编译再匹配

明天预告：Day 4 — OOP 深入 & 类型系统（metaclass 入门、描述器、ABC/Protocol、dataclass、类型注解）

Day 4 — OOP 深入 & 类型系统（完整版）

一、属性控制与内存优化

1. 是什么

Python 的属性控制远比”self.xxx = xxx”丰富。你可以在不改变外部接口的前提下，控制属性的读写行为、校验逻辑，甚至优化内存占用。

今天要学的：

工具	一句话	解决什么问题
property	用方法伪装成属性，读写时自动触发逻辑	不想破坏已有代码，又需要加校验/计算
__slots__	固定实例的属性名，去掉 __dict__	百万级实例时内存爆了
cached_property	算一次，缓存后不再重算	昂贵的计算结果反复访问

2. 解决了什么问题

问题 1：想要校验，但不想破坏外部接口

```
# 用户类，age 不能是负数
class User:
    def __init__(self, age):
        self.age = age # 直接赋值，负数也放行

u = User(-5) # ✕ 没人提醒

# 传统方案：改成 getter/setter 方法
class User:
    def set_age(self, value):
        if value < 0:
            raise ValueError("年龄不能为负")
        self._age = value
    def get_age(self):
        return self._age

u = User()
u.set_age(-5) # 可以校验了，但 u.age 不能用了

# 用 property：既保留 u.age 语法，又能校验
class User:
    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("年龄不能为负")
        self._age = value
```

问题 2：计算属性每次都重算

```
class DataReport:
    def __init__(self, data):
        self.data = data

    @property
    def summary(self):
        print("正在计算...", end=" ")
        return sorted(self.data)[:5] # 昂贵的排序

r = DataReport([3, 1, 4, 1, 5, 9, 2, 6, 5])
print(r.summary) # 正在计算...
print(r.summary) # 正在计算... 每次都重算!
```

用 `@cached_property` 只算一次。

问题 3：百万个实例，内存爆了

```
# 一个 Person 实例有 __dict__，约 56+ 字节
# 100万个人物 → 56MB
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# 用 __slots__ 固定属性 → 32 字节，省 43%
class Person:
```

```
__slots__ = ("name", "age")
def __init__(self, name, age):
    self.name = name
    self.age = age
```

3. 核心理论

property

一句话：用方法伪装成属性，读写触发自定义逻辑。

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("低于绝对零度！")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

t = Temperature(100)
print(t.fahrenheit)      # 212.0
t.fahrenheit = 32
print(t.celsius)         # 0.0
```

应用场景

- 校验：赋值时做类型/范围检查
- 计算属性：属性值来自其他字段计算
- 向后兼容：已有的 obj.field 改成方法但不改调用方
- 只读属性：只定义 @property 不定义 setter

注意：property 读写的性能比直接访问 self._xxx 略慢（方法调用开销）。热路径上谨慎使用。

cached_property

```
from functools import cached_property

class DataReport:
    def __init__(self, data):
        self.data = data

    @cached_property
    def summary(self):
        print("正在计算...")
        return sorted(self.data)[:5]

r = DataReport([3, 1, 4, 1, 5, 9, 2, 6])
```



```
print(r.summary) # □ 正在计算... [1, 1, 2, 3, 4]
print(r.summary) # ✓ 直接返回缓存, 不重算
```

什么时候用 - 计算结果不变（数据不修改） - 计算开销大 - 同一实例多次访问

什么时候不该用 - 数据会变化（缓存不会自动失效） - 内存敏感（计算结果会一直留着）

`__slots__`

一句话：固定属性名，去掉 `__dict__`，省内存 + 提速度。

```
class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
print(p.x)          # 1
# p.z = 3           # ✗ AttributeError
# print(p.__dict__) # ✗ 没有 __dict__

# 内存对比
import sys

class RegularPoint:
    def __init__(self, x, y):
        self.x = x
        self.y = y

r = RegularPoint(1, 2)
s = Point(1, 2)
print(sys.getsizeof(r)) # 56 bytes
print(sys.getsizeof(s)) # 32 bytes - 省 43%
```

有什么代价 - 不能动态添加属性（`a.new_attr = xxx` 会报错） - 不能给实例绑定方法（除非在 `__slots__` 中声明） - 继承时子类也必须定义 `__slots__` 才能利用内存优化 - 多重继承时可能有冲突

应用场景 - 数据类、DTO（数据传输对象）、Entity（领域实体） - 游戏中的坐标/向量/粒子系统（海量实例） - 配置类（属性固定）

二、接口与类型系统

1. 是什么

当代码规模增大，你需要一种方式来定义“什么对象可以做什么事”。Python 给了你三种选择：

工具	哲学	检查时机	典型场景
ABC (Abstract Base Class)	你继承我，才是我的人	实例化时（显式继承）	框架、库、需要默认实现
Protocol	你长这样，就是我的类型	类型检查时（结构匹配）	第三方类型适配、鸭子类型+类型安全
<code>__init_subclass__</code>	有人继承我，我知道	子类创建时自动触发	插件注册、自动发现

2. 解决了什么问题

问题 1：如何定义“这个类必须实现 xxx 方法”

```
# 没有任何约束，全靠自觉
class Circle:
    def area(self): return 3.14 * r ** 2

class Square:
    def area(self): return s ** 2

# 理想：规定所有 Shape 必须有 area()，漏了报错
```

ABC 解决这个问题：没实现 area() 的类根本创建不了实例。

问题 2：如何让第三方类也符合你的接口

```
# 你的框架要求对象有 draw() 方法
def render_all(objects):
    for obj in objects:
        obj.draw()

# 第三方库的图形类有 draw() 但不是你的子类
# → 应该能用，但类型检查会报错（如果有类型检查的话）
# → Protocol 解决：结构匹配，不需要继承
```

问题 3：如何自动发现所有插件

```
# 不想手动维护一个插件列表
class AudioPlugin: ...
class VideoPlugin: ...
class ImagePlugin: ...

# plugins = [AudioPlugin, VideoPlugin, ImagePlugin] # □ 每次加插件都要改这行

用 __init_subclass__：只要有人继承你的基类，自动注册。
```

3. 核心理论

ABC (Abstract Base Class)

一句话：定义接口契约——子类必须实现哪些方法，少一个都不让你实例化。

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

    def describe(self) -> str:
        return f"面积: {self.area()}, 周长: {self.perimeter()}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2
```

```

def perimeter(self) -> float:
    return 2 * 3.14159 * self.radius

# shape = Shape() # ✕ TypeError: Can't instantiate abstract class
circle = Circle(5)
print(circle.describe()) # 面积: 78.53975, 周长: 31.4159

```

ABC 注册第三方类

不想（或不能）让第三方类继承你的 ABC？注册一下就行：

```

@Shape.register
class ThirdPartyShape:
    def area(self):
        return 42
    def perimeter(self):
        return 0

print(isinstance(ThirdPartyShape(), Shape)) # True

```

ABC 还能干什么

- @abstractmethod、@abstractclassmethod、@abstractproperty
- 在 __init_subclass__ 里做自动检查
- 结合 ABCMeta 自定义元类行为

什么时候用 ABC - 框架/库定义核心接口（Django、Flask、FastAPI 都用） - 需要继承时自动获得默认实现 - 需要 isinstance 做运行时类型判断

Protocol

一句话：鸭子类型 + 类型检查——对象只要能做这件事就行，不需要是一家人。

```

from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> None: ...

class Circle:
    def draw(self) -> None:
        print("画个圆 ◯")

class Square:
    def draw(self) -> None:
        print("画个方 ■")

# Circle 和 Square 不需要继承 Drawable!
def render(obj: Drawable) -> None:
    obj.draw()

render(Circle()) # ✔ 类型检查通过
render(Square()) # ✔
# render(42) # ✕ mypy 会报错

```

runtime_checkable

默认 Protocol 只做静态类型检查。要让 isinstance 也能用：

```

from typing import Protocol, runtime_checkable

```

```
@runtime_checkable
class Iterable(Protocol):
    def __iter__(self):
        ...

print(isinstance([1, 2], Iterable)) # True
print(isinstance(42, Iterable))    # False
```

但注意：runtime_checkable 只检查方法存在，不检查签名是否正确。

ABC vs Protocol 怎么选

场景	ABC	Protocol
框架定义接口+需要默认实现 ✓ 用 ABC		✗ Protocol 没有默认实现
给第三方类加接口	✗ 要改源码	✓ 鸭子类型就行
需要 isinstance 检查	✓ 默认支持	✓ 加 @runtime_checkable
需要强制继承关系	✓ 显式	✗ 结构匹配
运行时性能	略慢	快
典型用途	Django Model, FastAPI Depends	函数参数约束、插件契约

__init_subclass__

一句话：只要有人继承了你的类，自动触发回调。

```
class PluginBase:
    _registry: dict[str, type] = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        PluginBase._registry[cls.__name__] = cls
        print(f"新插件注册: {cls.__name__}")

class AudioPlugin(PluginBase): pass
class VideoPlugin(PluginBase): pass
class ImagePlugin(PluginBase): pass

print(PluginBase._registry)
# {'AudioPlugin': ..., 'VideoPlugin': ..., 'ImagePlugin': ...}
```

经典应用：插件系统

```
class PluginBase:
    _registry = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        cls._order = kwargs.get("order", 100) # 支持排序
        PluginBase._registry[cls.__name__] = cls

    @classmethod
    def run_all(cls):
        """按 order 顺序执行所有插件"""
        for name, plugin_cls in sorted(
            cls._registry.items(),
            key=lambda x: x[1]._order
        ):
            plugin = plugin_cls()
            plugin.execute()

    def execute(self):
```

```
raise NotImplementedError
```

```
class LogPlugin(PluginBase, order=1):
    def execute(self):
        print("□ 日志插件运行")

class ValidatePlugin(PluginBase, order=2):
    def execute(self):
        print("✓ 验证插件运行")
```

```
PluginBase.run_all()
# □ 日志插件运行
# ✓ 验证插件运行
```

什么时候用 - 插件/扩展注册器 - 自动发现子类（替代入口文件手动 import） - 子类元信息收集（API 路由注册）

三、数据类与类型注解

1. 是什么

写一个“只有数据没有逻辑”的类，在 Python 中有很多方式。你写的样板代码越多，维护负担越大。

工具	一句话	缺点
普通 dict	最灵活，啥都能存	无类型约束，用 ['x'] 不如 .x 顺手
namedtuple	不可变的元组 + 字段名	不可变、不能加方法
dataclass	自动生成 __init__、__repr__、__eq__	3.7+ 才引入，有性能开销
类型注解	告诉 IDE 和工具这是什么类型	运行时不管，但工具链用

2. 解决了什么问题

问题：打个数据类要写一堆样板代码

```
class User:
    def __init__(self, name, email, age=0):
        self.name = name
        self.email = email
        self.age = age

    def __repr__(self):
        return f"User(name={self.name}, email={self.email}, age={self.age})"

    def __eq__(self, other):
        if not isinstance(other, User):
            return NotImplemented
        return (self.name, self.email, self.age) == (other.name, other.email, other.age)
```

30 行代码，就为了放 3 个字段

用 dataclass：

```
from dataclasses import dataclass
```

```
@dataclass
class User:
    name: str
```

```

    email: str
    age: int = 0

# __init__、__repr__、__eq__ 全自动
u = User("Leo", "leo@example.com")
print(u)  # User(name='Leo', email='leo@example.com', age=0)

```

3. 核心理论

dataclass

一句话：装饰器自动生成 `__init__`、`__repr__`、`__eq__`、`__hash__`，让你只关注数据字段。

基础用法

```

from dataclasses import dataclass, field, asdict, astuple

@dataclass
class User:
    name: str
    email: str
    age: int = field(default=0, repr=False)  # repr 中隐藏
    tags: list[str] = field(default_factory=list)  # 不能用 [] 做默认值！

    def __post_init__(self):
        """初始化后自动调用，适合做校验/转换"""
        self.email = self.email.lower()

u = User("Leo", "Leo@Example.com")
print(u.email)  # "leo@example.com"
print(u)  # User(name='Leo', email='leo@example.com')
print(asdict(u))  # {'name': 'Leo', 'email': 'leo@example.com', 'age': 0, 'tags': []}

```

重要细节：为什么不能写 `tags: list = []`

```

# ⚠ 所有实例共享同一个空列表！
@dataclass
class Bad:
    items: list = []

a = Bad()
b = Bad()
a.items.append("hack")
print(b.items)  # ['hack'] □ 互相污染了

# ✓ 用 default_factory
@dataclass
class Good:
    items: list = field(default_factory=list)

```

dataclass 进阶选项

```

@dataclass(frozen=True)  # 不可变（类似 namedtuple，但有方法）
@dataclass(order=True)  # 自动生成 __lt__、__gt__ 等比较方法
@dataclass(kw_only=True)  # 必须关键字传参（3.10+）
@dataclass(slots=True)  # __slots__ 优化（3.10+）

```

dataclass vs namedtuple vs dict

```

from dataclasses import dataclass
from collections import namedtuple
import sys

```

```
# dict — 灵活但无结构
d = {"x": 1, "y": 2}
print(sys.getsizeof(d)) # ~232 bytes

# namedtuple — 不可变、轻量
PointNT = namedtuple("PointNT", ["x", "y"])
p = PointNT(1, 2)
print(sys.getsizeof(p)) # ~64 bytes

# dataclass — 可变的 namedtuple 升级版
@dataclass(slots=True)
class PointDC:
    x: int
    y: int
p = PointDC(1, 2)
print(sys.getsizeof(p)) # ~32 bytes (用了 slots)
```

应用场景 - API 请求/响应模型 - 配置项 - DTO（数据传输对象） - 需要 `__eq__` 的值对象 - 替代 `namedtuple`（需要可变性时）

什么时候不该用 `dataclass` - 需要复杂的 `__init__` 行为（用普通 `class` + 自定义 `__init__`） - 性能极度敏感（`dataclass` 构造略慢于手写 `class`） - 3.7 以下的 Python（不支持）

类型注解实战

```
from typing import Optional, Union, Any, Callable, TypeVar
from collections.abc import Sequence, Mapping

# 基本注解
def greet(name: str) -> str:
    return f"Hello {name}"

# Optional / Union (= 3.10+ 可以用 str | None)
def find_user(id: int) -> Optional[dict]:
    ...

# Callable — 函数作为参数的类型
def process(
    items: list[int],
    callback: Callable[[int], str]
) -> list[str]:
    return [callback(x) for x in items]

# TypeVar — 泛型
T = TypeVar("T")
def first(items: Sequence[T]) -> T | None:
    return items[0] if items else None

# 运行时不会强制检查，但 IDE + mypy 会
```

今日练习

1. **property 做校验**：实现一个 `EmailField` 类，用 `property` 确保 email 包含 @，赋值时自动校验
2. **ABC 做插件框架**：用 `ABC` + `__init_subclass__` 写一个插件注册器，支持插件按优先级排序执行
3. **Protocol 实战**：定义 `Comparable Protocol`（实现 `__lt__`），让 `sorted()` 函数类型安全地接受自定义对象
4. **dataclass 转 JSON**：用 `asdict()` 把嵌套 `dataclass` 递归转成 JSON，支持 `list[dataclass]`、`dict[str, dataclass]` 的嵌套场景

明天预告：Day 5 — 并发编程入门（GIL、threading、multiprocessing、asyncio 基础、concurrent.futures）

Day 5 — 并发编程入门（完整版）

一、理解并发：为什么 Python 的并发不一样

1. 是什么

并发（Concurrency）是让程序“同时”做多件事的能力。但 Python 里有两个陷阱：

概念	一句话	能不能加速 CPU 密集任务
多线程 (threading)	多个执行流交替跑，共用内存	✗ 因为 GIL
多进程 (multiprocessing)	多个解释器隔离跑，各自独立	✓ 真并行
异步 I/O (asyncio)	一个线程里切换任务，等 I/O 时干别的	✗ 单线程
并发 (concurrent.futures)	上面三者的高级封装	看底层选谁

关键前提：GIL（全局解释器锁）

CPython 解释器设计上的一个限制——同一时刻只有一个线程在执行 Python 字节码。

```
import threading, time
```

```
def count(n):
    while n > 0:
        n -= 1
```

```
# 单线程
```

```
start = time.time()
count(50_000_000)
print(f"单线程: {time.time() - start:.2f}s")
```

```
# 两线程 — 比单线程还慢（加锁开销）
```

```
t1 = threading.Thread(target=count, args=(25_000_000,))
t2 = threading.Thread(target=count, args=(25_000_000,))
start = time.time()
t1.start(); t2.start()
t1.join(); t2.join()
print(f"两线程: {time.time() - start:.2f}s")
```

2. 解决了什么问题

选错了并发方案，性能反而下降。

任务类型	举例	正确方案	错误方案
CPU 密集	计算、排序、图像处理	multiprocessing	threading ✗
I/O 密集	网络请求、文件读写、数据库	asyncio / threading	—
混合型	计算+等待交替	组合方案	—

```
# ✗ 错误：CPU 密集用 threading
```

```
def cpu_heavy():
    total = 0
    for i in range(10_000_000):
```



```
    total += i ** 2
    return total
```

```
t1 = threading.Thread(target=cpu_heavy) # GIL 锁死，白忙活
```

```
# ✓ 正确：CPU 密集用 multiprocessing
from multiprocessing import Pool
with Pool() as pool:
    results = pool.map(cpu_heavy, range(4))
```

3. 选择策略

你的场景	用这个
大量计算，要多核加速	multiprocessing / ProcessPoolExecutor
高并发网络请求/API 调用	asyncio + aiohttp
I/O 密集但代码已是同步的	ThreadPoolExecutor (简单改造)
需要和已有同步库配合	ThreadPoolExecutor
追求极致性能 + 新项目	asyncio
不想动脑子	concurrent.futures (统一接口)

二、threading — 多线程

1. 是什么

Python 标准库提供的线程编程接口。一个进程内的多个线程共享内存，适合 I/O 等待场景。

2. 解决了什么问题

不让程序在等待 I/O 时傻站着。

```
# ✓ 多线程：三个同时等
threads = [
    threading.Thread(target=download, args=(f"url{i}",))
    for i in range(3)
]
start = time.time()
for t in threads: t.start()
for t in threads: t.join()
print(f"多线程耗时: {time.time() - start:.1f}s") # ~2s
```

3. 核心理论

```
import threading, time

def worker(name, delay):
    print(f"{name} 启动")
    time.sleep(delay)
    print(f"{name} 完成")

# 创建并启动
threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(f"Worker-{i}", i))
    threads.append(t)
    t.start()
```

```
# 等待全部完成
for t in threads:
    t.join()

# 守护线程（主线程退出时自动结束）
t = threading.Thread(target=worker, args=("Daemon", 10), daemon=True)
t.start()
```

同步工具

同步工具	作用	场景
Lock	互斥锁，一次一个线程	保护共享数据
RLock	可重入锁，同一线程可多次 lock	递归函数中加锁
Semaphore	信号量，限制 N 个线程同时访问	限流、连接池
Event	事件标志，一个线程发信号 N 个等	等待条件满足
Condition	条件变量，更复杂的事件通知	生产者-消费者

```
# ✗ 不加锁：累加不是原子操作
counter = 0
def increment():
    global counter
    for _ in range(100_000):
        counter += 1 # 读→改→写，三条指令！
```

```
# ✓ 加锁
class SafeCounter:
    def __init__(self):
        self.value = 0
        self._lock = threading.Lock()

    def increment(self):
        with self._lock:
            self.value += 1
```

```
# RLock — 可重入
lock = threading.RLock()
def recursive(n):
    with lock:
        if n > 0:
            recursive(n - 1) # 不会死锁
```

```
# Event — 信号等待
ready = threading.Event()
def waiter():
    ready.wait() # 阻塞直到 set
    print("开冲！")

def setter():
    time.sleep(2)
    ready.set()
```

```
# Semaphore — 限制并发
sem = threading.Semaphore(3)
def limited():
    with sem:
        print(f"允许并发")
```

4. 生产者-消费者模式

```
import queue

q = queue.Queue(maxsize=10)

def producer():
    for i in range(5):
        q.put(f"item-{i}")
        print(f"生产: item-{i}")
    q.put(None)  # 哨兵信号

def consumer():
    while True:
        item = q.get()
        if item is None:
            break
        print(f"消费: {item}")
        q.task_done()
```

5. ThreadPoolExecutor — 高级封装

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_url(url):
    time.sleep(1)
    return f"数据来自 {url}"

urls = [f"http://api.example.com/{i}" for i in range(10)]

with ThreadPoolExecutor(max_workers=5) as executor:
    # submit — 逐个提交
    futures = {executor.submit(fetch_url, url): url for url in urls}
    for f in as_completed(futures):
        print(f.result())

    # map — 批量提交（保持顺序）
    results = list(executor.map(fetch_url, urls))
```

三、multiprocessing — 多进程

1. 是什么

绕过 GIL 的唯一方式——启动多个 Python 进程，每个有自己独立的解释器和内存空间。

2. 解决了什么问题

CPU 密集任务真正用上多核。

3. 核心理论

```
from multiprocessing import Process

def cpu_heavy(n):
    total = 0
    for i in range(n):
        total += i ** 2
    return total

p = Process(target=cpu_heavy, args=(10_000_000,))
```

```
p.start()
p.join()
```

ProcessPoolExecutor

```
from multiprocessing import Pool, cpu_count
from concurrent.futures import ProcessPoolExecutor

# Pool (经典 API)
with Pool(processes=cpu_count()) as pool:
    results = pool.map(cpu_heavy, [10_000_000] * 4)

# ProcessPoolExecutor (统一接口, 推荐)
with ProcessPoolExecutor(max_workers=cpu_count()) as executor:
    results = list(executor.map(cpu_heavy, [10_000_000] * 4))
```

进程间通信

通信方式	速度	复杂度	场景
Queue	中等	低	生产者-消费者
Pipe	快	中	双向通信
Value / Array	快	低	简单共享数据
shared_memory	最快	高	大块数据共享

```
from multiprocessing import Queue, Pipe, Value, Array

# Queue
q = Queue()
q.put("data")
item = q.get()

# Pipe
parent_conn, child_conn = Pipe()
parent_conn.send(42)
child_conn.recv() # 42

# 共享内存
counter = Value("i", 0) # int
counter.value += 1
arr = Array("d", [1.0, 2.0, 3.0]) # double[]
```

⚠ multiprocessing 的坑：

1. Windows 上需要 if __name__ == "__main__" 保护
2. 进程间不能共享普通 Python 对象（需要序列化）
3. 启动开销大（每个进程要 import 全部模块）
4. 调试困难（多进程，日志/异常可能丢失）
5. 资源隔离：一个进程 crash 不影响其他进程

四、asyncio — 异步 I/O

1. 是什么

用一个线程、一个事件循环，在等待 I/O 时自动切换到其他任务。**不是并行，是并发。**

时间轴 →

线程: [task1] 等待I/O [task2] 等待I/O [task1]...

↓ 切换 ↓ 切换

2. 解决了什么问题

海量 I/O 密集任务的高效方案。

3. 核心概念

概念	一句话	类似 threading 的什么
async def	定义一个协程函数	类似 Thread(target=...)
await	交出控制权，等结果回来再继续	类似 t.join()
asyncio.run()	启动事件循环	类似 t.start()
asyncio.create_task()	把协程注册到事件循环	类似创建线程
asyncio.gather()	并发跑多个协程	类似 t.join() 多个

4. 核心理论

```
import asyncio

async def fetch_data(url):
    print(f"请求 {url}")
    await asyncio.sleep(1)  # 模拟 I/O 等待
    print(f"完成 {url}")
    return {"url": url, "data": "..."}

# 方式1: 单个协程
result = asyncio.run(fetch_data("http://example.com"))

# 方式2: 并发执行
async def main():
    tasks = [
        asyncio.create_task(fetch_data(f"url-{i}"))
        for i in range(3)
    ]
    results = await asyncio.gather(*tasks)
    return results

results = asyncio.run(main())
```

规则

```
# ✗ await 只能在 async def 内
def wrong():
    await asyncio.sleep(1)  # SyntaxError!

# ✓ 正确
async def correct():
    await asyncio.sleep(1)
```

I/O 密集 ✓ vs CPU 密集 ✗

```
# ✓ I/O 密集 — 最佳场景
async def fetch_all():
    import aiohttp
    async with aiohttp.ClientSession() as session:
```

```
tasks = [session.get(url) for url in urls]
responses = await asyncio.gather(*tasks)
return [await r.json() for r in responses]
```

```
# ✕ CPU 密集 — 会阻塞事件循环
async def compute():
    for i in range(10_000_000):
        pass # 一直不 await, 事件循环卡死!
    return 42
```

混合方案：run_in_executor

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

async def main():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool,
            blocking_io_operation, # 同步函数
            arg1, arg2
        )
```

同步原语

```
sem = asyncio.Semaphore(3) # 限制并发数
lock = asyncio.Lock()     # 互斥
q = asyncio.Queue()        # 队列
```

```
async def fetch_with_limit():
    async with sem:
        await fetch_url(url)
```

```
# 超时控制
```

```
try:
    result = await asyncio.wait_for(
        fetch_url(url),
        timeout=5.0
    )
except asyncio.TimeoutError:
    print("请求超时")
```

5. 完整例子：并发下载器

```
import asyncio
import aiohttp

async def download_one(session, url):
    async with session.get(url) as resp:
        content = await resp.read()
        filename = url.split("/")[-1]
        with open(filename, "wb") as f:
            f.write(content)
    return filename

async def download_many(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [download_one(session, url) for url in urls]
        return await asyncio.gather(*tasks)

urls = [
```

```

    "https://example.com/file1.jpg",
    "https://example.com/file2.jpg",
]
results = asyncio.run(download_many(urls))

```

今日练习

1. **ThreadPoolExecutor 下载器**：用 ThreadPoolExecutor 并发下载多张图片
2. **asyncio 爬虫**：用 aiohttp 并发请求 10 个 API，加超时 + 错误处理 + 重试
3. **多进程计算**：用 ProcessPoolExecutor 并行计算多个大数的质因数分解
4. **综合练习**：先用 asyncio 下载文件，再用 ProcessPoolExecutor 处理图片（缩略图生成）

明天预告：Day 6 — 工程化实战（pytest、CLI 工具、项目结构、SQLAlchemy、FastAPI）

Day 6 — 工程化实战（完整版）

一、项目结构 — 从一开始就做对

1. 是什么

Python 项目没有“官方”目录结构。但社区总结出了最佳实践——**src 布局**。

```

my-project/
├── src/
│   └── my_package/
│       ├── __init__.py    # 包标记 + 公共导出
│       ├── __main__.py    # python -m my_package 入口
│       ├── cli.py
│       ├── models.py
│       └── utils.py
├── tests/
│   ├── __init__.py
│   ├── test_models.py
│   └── test_cli.py
├── pyproject.toml        # 现代包配置入口
├── README.md
├── .gitignore
└── Dockerfile

```

2. 解决了什么问题

不用 src 布局，测试时可能 import 了错误路径：

```

# 项目根目录下直接放 my_package/
# 测试时 import my_package 可能 import 了项目目录（没安装的版本）
# 而不是 pip install 好的正式版本

```

```

# src 布局强迫你先 pip install -e . 安装包
# 测试环境和用户环境一致 → 不会再出现“我机器上能跑”

```

3. pyproject.toml

现代 Python 包配置——取代了 setup.py、setup.cfg 和 requirements.txt。

```
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "my-cli-tool"
version = "0.1.0"
description = "一个有用的 CLI 工具"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.31",
    "click>=8.1",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "ruff>=0.5",
    "mypy>=1.10",
]

[project.scripts]
my-cli = "my_package.cli:main" # 安装后命令行直接敲 my-cli

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100

pip install -e . # 开发模式安装（可编辑）
pip install -e ".[dev]" # 带开发依赖
```

文件	作用	还常见吗
pyproject.toml	一切配置的入口	✔ 标准
setup.py	复杂构建逻辑	⚠ 部分项目还有
setup.cfg	旧式声明式配置	✗ 已被 toml 取代
requirements.txt	精确锁定依赖版本	✔ 部署用

二、pytest — 专业测试框架

1. 是什么

pytest 是 Python 最流行的测试框架。核心哲学：用断言代替样板代码。

特性	unittest	pytest
测试用例	必须继承 TestCase	普通函数就行
断言	self.assertEqual(a, b)	assert a == b
fixture	setUp / tearDown	@pytest.fixture
参数化	需要子类	@pytest.mark.parametrize
插件生态	一般	丰富（pytest-cov, pytest-mock）

2. 对比


```
# unittest 方式
import unittest
class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(1 + 1, 2)

# pytest 方式 - 就是普通函数
def test_add():
    assert 1 + 1 == 2
```

3. 核心 API

pytest	# 自动发现 test_*.py / *_test.py
pytest -v	# 详细输出
pytest -x	# 第一个失败就停
pytest --pdb	# 失败进调试器
pytest -k "add or dict"	# 按名字过滤
pytest test_math.py::test_add	# 只跑指定测试
pytest -s	# 显示 print 输出
pytest --cov=my_package	# 覆盖率报告
pytest -v --durations=5	# 显示最慢的 5 个测试
pytest --lf	# 只跑上次失败的
pytest --ff	# 先跑上次失败的，再跑剩下的

Fixture

```
import pytest

@pytest.fixture
def temp_dir():
    """创建临时目录，测试完自动清理"""
    import tempfile
    with tempfile.TemporaryDirectory() as tmp:
        yield tmp # yield 之前的代码 = setUp, 之后的 = tearDown

@pytest.fixture
def sample_data():
    """返回测试数据（无副作用）"""
    return {"name": "Leo", "age": 30}

# fixture 作用域
@pytest.fixture(scope="session") # 一次测试会话只创建一次
@pytest.fixture(scope="module") # 每个模块一次
@pytest.fixture(scope="class") # 每个类一次
@pytest.fixture(scope="function") # 每个测试函数一次（默认）

def test_file_creation(temp_dir):
    path = os.path.join(temp_dir, "test.txt")
    with open(path, "w") as f:
        f.write("hello")
    assert os.path.exists(path)
```

参数化测试

```
@pytest.mark.parametrize("input, expected", [
    (1, 1),
    (2, 4),
    (3, 9),
    (10, 100),
])
def test_square(input, expected):
    assert input ** 2 == expected
```

```
# 多参数组合（笛卡尔积）
@pytest.mark.parametrize("x", [1, 2])
@pytest.mark.parametrize("y", [10, 20])
def test_combine(x, y):
    assert x + y > 0 # 跑 4 次: (1,10), (1,20), (2,10), (2,20)
```

Mock

```
from unittest.mock import Mock, patch

# Mock 一个对象
mock_response = Mock()
mock_response.json.return_value = {"status": "ok"}
mock_response.status_code = 200

# patch 替换真实函数
with patch("requests.get", return_value=mock_response):
    result = my_function()
    assert result["status"] == "ok"

# patch 也可以作为装饰器
@patch("requests.get")
def test_api(mock_get):
    mock_get.return_value.json.return_value = {"success": True}
    result = call_api()
    assert result["success"]
```

conftest.py

```
# tests/conftest.py — 自动被所有测试文件共享
import pytest
from my_package import create_app

@pytest.fixture
def app():
    app = create_app()
    yield app

@pytest.fixture
def client(app):
    return app.test_client()

# tests/test_routes.py — 直接用
def test_home(client):
    resp = client.get("/")
    assert resp.status_code == 200
```

三、CLI 工具 — Click / Typer

1. 是什么

Click 是 Python 最成熟的命令行工具库。

2. 解决了什么问题

不用再手写 sys.argv 解析。

```
# ✖ 手工解析
if len(sys.argv) < 2:
    print("Usage: ...")
    sys.exit(1)
name = sys.argv[1]

# ✔ Click
import click

@click.command()
@click.argument("name")
@click.option("--count", default=1, help="问候次数")
@click.option("--upper/--no-upper", default=False)
@click.option("--greeting", default="Hello", show_default=True)
def greet(name, count, upper, greeting):
    """问候某人。"""
    for _ in range(count):
        msg = f"{greeting} {name}"
        if upper:
            msg = msg.upper()
        click.echo(msg)

if __name__ == "__main__":
    greet()

python greet.py Leo --count 3 --upper
python greet.py --help
# 自动生成:
# Usage: greet.py [OPTIONS] NAME
```

多命令分组

```
@click.group()
def cli():
    """项目管理工具"""
    pass

@cli.command()
def init():
    """初始化项目"""
    click.echo("项目已初始化 ✔")

@cli.command()
@click.argument("name")
def create(name):
    """创建新项目"""
    click.echo(f"已创建 {name} ✔")

@cli.command()
@click.option("--force", is_flag=True)
def clean(force):
    """清理项目"""
    if force:
        click.confirm("确认删除?", abort=True)
    click.echo("已清理 ✔")

if __name__ == "__main__":
    cli()

python cli.py init
python cli.py create my-app
python cli.py clean --force
python cli.py --help # 自动列出所有命令
```

Click 特性	写法	效果
Argument	@click.argument("name")	必填位置参数
Option	@click.option("--count")	可选命名参数
Flag	@click.option("--verbose/--quiet")	布尔开关
确认	click.confirm("确认? ")	交互确认
选择	click.Choice(["dev", "prod"])	限制可选值
密码	click.prompt("Password", hide_input=True)	隐藏输入

四、FastAPI 入门

1. 是什么

FastAPI 是目前 Python 增长最快的 Web 框架。核心卖点：**基于类型注解的自动化**。

2. 解决了什么问题

方法签名、数据校验、文档三件事应该同步，而不是各写各的。

```
# Flask 方式：手写校验，手写文档
@app.route("/items/<int:item_id>")
def get_item(item_id):
    if not isinstance(item_id, int):
        return {"error": "id must be int"}, 400

# FastAPI 方式：类型注解 = 校验 + 文档
@app.get("/items/{item_id}")
def get_item(item_id: int): # 自动校验类型，生成 OpenAPI 文档
    ...
```

3. 核心理论

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="我的 API")

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True

items: dict[int, Item] = {}
next_id = 1

@app.post("/items", status_code=201)
def create_item(item: Item):
    global next_id
    items[next_id] = item
    item_id = next_id
    next_id += 1
    return {"id": item_id, **item.model_dump()}

@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id not in items:
        raise HTTPException(404, "物品未找到")
```

```

        return {"id": item_id, **items[item_id].model_dump()}

@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return list(items.values())[skip:skip + limit]

pip install fastapi uvicorn
uvicorn app:app --reload
# http://localhost:8000/docs — 自动 Swagger 文档!

```

4. Pydantic 进阶

```

from pydantic import BaseModel, Field, HttpUrl, EmailStr

class User(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    email: str
    age: int = Field(ge=0, le=150)
    website: HttpUrl | None = None

    @field_validator("email")
    @classmethod
    def email_must_contain_at(cls, v):
        if "@" not in v:
            raise ValueError("邮箱必须包含 @")
        return v.lower()

```

5. 依赖注入

```

from fastapi import Depends, Header

def get_db():
    db = Database()
    try:
        yield db
    finally:
        db.close()

def verify_token(token: str = Header(...)):
    if token != "my-secret":
        raise HTTPException(401)

@app.get("/items")
def list_items(
    db = Depends(get_db),
    token = Depends(verify_token),
):
    return db.query_all()

```

FastAPI 特性	作用	代码量节省
类型注解 = 校验	自动验证请求参数	50%
类型注解 = 文档	自动 OpenAPI + Swagger UI	100%
Pydantic	请求/响应体校验	60%
Depends	依赖注入	40%
自动序列化	dict → JSON	30%

五、SQLAlchemy — 数据库 ORM

1. 是什么

SQLAlchemy 是 Python 最强大的 ORM（对象关系映射）。2.0 版本大幅简化了 API。

2. 解决了什么问题

```
# ✕ 手写 SQL
cursor.execute("SELECT * FROM items WHERE price > ?", (500,))
rows = cursor.fetchall()
for row in rows:
    print(f"Item: {row[0]}, Price: {row[3]}") # 魔法下标

# ✓ SQLAlchemy: 操作 Python 对象
items = session.query(Item).filter(Item.price > 500).all()
for item in items:
    print(f"Item: {item.name}, Price: {item.price}") # 属性名
```

3. 核心理论

```
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.orm import declarative_base, Session
```

```
engine = create_engine("sqlite:///items.db", echo=True)
Base = declarative_base()
```

```
class Item(Base):
    __tablename__ = "items"
    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    price = Column(Float, default=0.0)
    created_at = Column(DateTime)
```

```
Base.metadata.create_all(engine) # 自动建表
```

```
# Create
with Session(engine) as session:
    item = Item(name="笔记本电脑", price=999.99)
    session.add(item)
    session.commit()
    session.refresh(item)
    print(item.id)
```

```
# Read
item = session.get(Item, 1)
expensive = session.query(Item).filter(Item.price > 500).all()
cheap = session.query(Item).filter_by(price=0).all()
```

```
# 高级查询
from sqlalchemy import desc, func
session.query(Item).filter(
    Item.name.like("%笔记本%")
).order_by(desc(Item.price)).limit(10).all()
```

```
# Update
item = session.get(Item, 1)
item.price = 899.99
session.commit()
```

```
# Delete
session.delete(item)
session.commit()
```

4. 关联查询

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    items = relationship("Item", back_populates="owner")

class Item(Base):
    __tablename__ = "items"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    owner_id = Column(Integer, ForeignKey("users.id"))
    owner = relationship("User", back_populates="items")

# 关联查询
user = session.get(User, 1)
print(user.items)          # 自动关联
item = session.get(Item, 1)
print(item.owner.name)     # 反向关联
```

今日练习

1. 用 pyproject.toml + src 布局创建新项目，配好 pytest 和 ruff
2. 用 Click 写一个 TODO 管理工具（add, list, done, delete），数据存 JSON
3. 用 FastAPI 把 TODO 工具变成 REST API，Pydantic 做校验
4. 用 pytest（fixture + parametrize + mock）给 TODO API 写完整测试

明天预告：Day 7 — 实战项目（从零搭建 URL 短链服务，综合所有知识）

Day 7 — 实战项目：URL 短链服务（完整版）

一、项目规划

1. 是什么

今天不是学新知识，而是用之前 6 天学到的所有东西，从零搭建一个可以部署的 Web 服务。

我们做一个 **URL Shortener（短链服务）**——把长网址变成短码。

功能清单：

- POST /shorten — 提交长网址，返回短码
- GET /{short_code} — 访问短码，301 重定向到原网址
- GET /stats/{code} — 查看短码的访问统计
- CLI 管理工具 — 初始化数据库、列出短链、删除短链

2. 技术栈速览

技术	之前哪天学的	这里用来干嘛
FastAPI	Day 6	API 框架
Pydantic	Day 4	数据校验
SQLAlchemy	Day 6	数据库 ORM
Click	Day 6	CLI 管理工具
pytest	Day 6	测试
logging	Day 3	日志
secrets	Day 1	安全随机短码

3. 项目结构

```

url-shortener/
├── src/
│   └── shortener/
│       ├── __init__.py
│       ├── __main__.py      # python -m shortener
│       ├── app.py           # FastAPI 应用
│       ├── models.py        # SQLAlchemy 模型
│       ├── schemas.py       # Pydantic schema
│       ├── database.py      # 数据库连接
│       ├── cli.py           # Click 管理命令
│       └── utils.py          # 工具函数
├── tests/
│   ├── __init__.py
│   ├── test_api.py
│   └── conftest.py
├── pyproject.toml
└── .gitignore

```

先搭好骨架再填充代码。这就是 Day 6 讲的 src 布局。

二、数据库层 — SQLAlchemy

```

# src/shortener/models.py
from sqlalchemy import Column, Integer, String, DateTime, BigInteger, func
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class URL(Base):
    __tablename__ = "urls"

    id = Column(Integer, primary_key=True)
    original_url = Column(String(2048), nullable=False)
    short_code = Column(String(10), unique=True, index=True, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
    visits = Column(BigInteger, default=0)

# src/shortener/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///shortener.db")
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

```



```
def get_db():
    """FastAPI 依赖注入用的函数。每次请求获取一个新 session。"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

设计决策解析： - short_code 设了 unique=True + index=True：查找短码是核心操作，必须快 - BigInteger 存 visits：日活高的链接访问量可能很大 - server_default=func.now()：数据库自动填时间，应用层不用管 - 用 DATABASE_URL 环境变量：随时切换 SQLite → PostgreSQL，代码不用改

三、数据校验层 — Pydantic

```
# src/shortener/schemas.py
from pydantic import BaseModel, HttpUrl
from datetime import datetime

class ShortenRequest(BaseModel):
    """创建短链的请求体"""
    url: HttpUrl # Pydantic 自动校验 URL 格式

class ShortenResponse(BaseModel):
    """创建短链的响应体"""
    short_code: str
    short_url: str
    original_url: str

class URLStats(BaseModel):
    """短链统计的响应体"""
    short_code: str
    original_url: str
    visits: int
    created_at: datetime
```

为什么用三个 Schema 而不是一个： - 请求和响应需要的字段不同 - HttpUrl 类型自动校验：传入的不是合法 URL 直接 422，不用手写正则 - 各管各的，修改一个不影响其他

四、工具函数层

```
# src/shortener/utils.py
import secrets
import string

def generate_short_code(length: int = 6) -> str:
    """生成安全随机的短码"""
    alphabet = string.ascii_lowercase + string.digits
    return "".join(secrets.choice(alphabet) for _ in range(length))

def generate_unique_code(db_session, length=6):
    """生成不重复的短码"""
    from .models import URL

    while True:
        code = generate_short_code(length)
        exists = db_session.query(URL).filter(
            URL.short_code == code
        ).first()
```

```

    if not exists:
        return code

```

设计决策解析： - secrets.choice 替代 random.choice：密码学安全的随机数，不能被人猜出下一个短码 - string.ascii_lowercase + digits（36 个字符）：6 位有 $36^6 \approx 21$ 亿个组合 - 循环直到找到不重复的码：碰撞概率极低，实际基本一次命中

五、应用入口 — FastAPI

```

# src/shortener/app.py
import logging
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.responses import RedirectResponse
from sqlalchemy.orm import Session

from .models import Base, URL
from .schemas import ShortenRequest, ShortenResponse, URLStats
from .database import engine, get_db
from .utils import generate_unique_code

logger = logging.getLogger(__name__)

# 启动时自动建表
Base.metadata.create_all(bind=engine)

app = FastAPI(title="URL Shortener")

@app.post("/shorten", response_model=ShortenResponse, status_code=201)
def shorten_url(
    request: ShortenRequest,
    db: Session = Depends(get_db),
):
    """提交长网址，返回短码"""
    short_code = generate_unique_code(db)
    db_url = URL(
        original_url=str(request.url),
        short_code=short_code,
    )
    db.add(db_url)
    db.commit()
    db.refresh(db_url)
    logger.info(f"新短链: {short_code} → {request.url}")
    return ShortenResponse(
        short_code=short_code,
        short_url=f"http://short.url/{short_code}",
        original_url=str(request.url),
    )

@app.get("/{short_code}")
def redirect_to_url(
    short_code: str,
    db: Session = Depends(get_db),
):
    """访问短链 → 301 重定向"""
    url_entry = db.query(URL).filter(URL.short_code == short_code).first()
    if not url_entry:
        raise HTTPException(status_code=404, detail="短链不存在")
    url_entry.visits += 1
    db.commit()
    return RedirectResponse(url=url_entry.original_url, status_code=301)

```

```
@app.get("/stats/{short_code}", response_model=URLStats)
def get_stats(
    short_code: str,
    db: Session = Depends(get_db),
):
    """查看短链统计"""
    url_entry = db.query(URL).filter(URL.short_code == short_code).first()
    if not url_entry:
        raise HTTPException(status_code=404, detail="短链不存在")
    return URLStats(
        short_code=url_entry.short_code,
        original_url=url_entry.original_url,
        visits=url_entry.visits,
        created_at=url_entry.created_at,
    )
```

逐行解析：

代码	作用	为什么这样写
Base.metadata.create_all(bind=engine)	自动建表	开发时省一个步骤；生产要用迁移工具
response_model=ShortenResponse	FastAPI 自动序列化并校验响应	确保响应结构正确，自动生成 API 文档
Depends(get_db)	依赖注入，每次请求创建/关闭 session	确保 session 正确关闭；测试可替换
status_code=201	创建资源返回 201	RESTful 规范
RedirectResponse(status_code=301)	301 永久重定向	浏览器会缓存，下次直接跳转

六、CLI 管理工具 — Click

```
# src/shortener/cli.py
import click
from .database import SessionLocal, engine
from .models import Base, URL
```

```
@click.group()
def cli():
    """URL Shortener 管理工具"""
    pass
```

```
@cli.command()
def init_db():
    """初始化数据库（建表）"""
    Base.metadata.create_all(bind=engine)
    click.echo("数据库已初始化 ✓")
```

```
@cli.command()
def list_urls():
    """列出所有短链"""
    db = SessionLocal()
    urls = db.query(URL).all()
    if not urls:
        click.echo("暂无短链")
    db.close()
    return
```

```

click.echo(f"{'短码':<12s} {'访问数':<8s} {'原始 URL'}")
click.echo("-" * 60)
for url in urls:
    click.echo(f"{url.short_code:<12s} {url.visits:<8d} {url.original_url}")
db.close()

```

```

@cli.command()
@click.argument("short_code")
def delete(short_code):
    """删除指定短链"""
    db = SessionLocal()
    url = db.query(URL).filter(URL.short_code == short_code).first()
    if not url:
        click.echo(f"未找到: {short_code}")
        db.close()
        return
    db.delete(url)
    db.commit()
    db.close()
    click.echo(f"已删除 {short_code} ✓")

```

```

if __name__ == "__main__":
    cli()

```

```

# 注册为命令行工具（在 pyproject.toml 中配置）
shortener init-db      # 建表
shortener list-urls    # 列出所有
shortener delete abc123 # 删除

```

七、pyproject.toml + main.py

```

[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

```

```

[project]
name = "url-shortener"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.110",
    "uvicorn>=0.27",
    "sqlalchemy>=2.0",
    "pydantic>=2.5",
    "click>=8.1",
]

```

```

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "httpx>=0.27",
    "ruff>=0.5",
]

```

```

[project.scripts]
shortener = "shortener.cli:cli"

```

```

[tool.setuptools.packages.find]
where = ["src"]

```

```
# src/shortener/__main__.py
# 支持 python -m shortener 启动
from .cli import cli
cli()
```

八、测试

```
# tests/conftest.py
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from fastapi.testclient import TestClient
from shortener.models import Base
from shortener.database import get_db
from shortener.app import app
```

```
@pytest.fixture
def db_session():
    """每个测试用独立的内存数据库"""
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(bind=engine)
    TestingSession = sessionmaker(bind=engine)
    session = TestingSession()
    yield session
    session.close()
```

```
@pytest.fixture
def client(db_session):
    """替换 FastAPI 的数据库依赖为测试数据库"""
    def override_get_db():
        yield db_session
    app.dependency_overrides[get_db] = override_get_db
    return TestClient(app)
```

```
# tests/test_api.py
import pytest
from datetime import datetime
from shortener.models import URL
```

```
class TestShortenURL:
    def test_shorten_returns_201_and_short_code(self, client):
        resp = client.post("/shorten", json={
            "url": "https://example.com/very/long/path"
        })
        assert resp.status_code == 201
        data = resp.json()
        assert "short_code" in data
        assert len(data["short_code"]) == 6

    def test_shorten_invalid_url_returns_422(self, client):
        resp = client.post("/shorten", json={
            "url": "not-a-url"
        })
        assert resp.status_code == 422
```

```
class TestRedirect:
    def test_redirect_returns_301(self, client):
        resp = client.post("/shorten", json={"url": "https://example.com"})
        code = resp.json()["short_code"]
```

```

    resp = client.get(f"/{code}", follow_redirects=False)
    assert resp.status_code == 301
    assert resp.headers["location"] == "https://example.com"

```

```

def test_nonexistent_code_returns_404(self, client):
    resp = client.get("/nonexist")
    assert resp.status_code == 404

```

```

class TestStats:
    def test_stats_tracks_visits(self, client):
        resp = client.post("/shorten", json={"url": "https://example.com"})
        code = resp.json()["short_code"]
        client.get(f"/{code}")
        client.get(f"/{code}")
        resp = client.get(f"/stats/{code}")
        assert resp.status_code == 200
        data = resp.json()
        assert data["visits"] == 2
        assert data["original_url"] == "https://example.com"

```

测试策略： - 用内存 SQLite (:memory:) 而不是真正的数据库文件 - 每个测试独立的 session，互不干扰
 - 通过 FastAPI 的 `dependency_overrides` 机制替换真实数据库

```

pytest -v                                # 全部跑通
pytest --cov=shortener                   # 带覆盖率

```

九、运行

```

# 1. 安装 (开发模式)
pip install -e ".[dev]"

# 2. 初始化数据库
shortener init-db

# 3. 启动 API
uvicorn shortener.app:app --reload --port 8000
# http://localhost:8000/docs ← 自动 Swagger!

# 4. 测试
pytest -v --cov=shortener

```

十、扩展方向

到这里你已经有了一个可运行的短链服务。以下是真实世界需要但本课程没展开的：

功能	怎么做	涉及的新技术
自定义短码	支持用户指定 code	加一个可选的 code 字段
过期时间	短链到期自动 404	加 <code>expires_at</code> 字段 + 定时清理
API Key 认证	请求头校验	FastAPI Header + Depends
异步改造	async handler + async SQLAlchemy	async/await + asyncpg
Docker 部署	容器化	Dockerfile + docker-compose
前端页面	HTML 表单创建短链	Jinja2 模板或纯前端
数据库迁移	安全修改表结构	Alembic

恭喜完成 7 天 Python 训练营！

你已经覆盖了：

Day	主题	核心工具
Day 1	基础语法速通	数据类型、函数、OOP、异常
Day 2	进阶语法	装饰器、生成器、上下文管理器
Day 3	标准库兵器谱	collections, itertools, logging, re
Day 4	OOP & 类型系统	property, ABC, Protocol, dataclass
Day 5	并发编程	threading, multiprocessing, asyncio
Day 6	工程化	pytest, FastAPI, SQLAlchemy, Click
Day 7	实战项目	URL Shortener 全栈

下一步建议： - 把这个项目真正部署到服务器上 - 读开源项目源码（FastAPI、uvicorn 都是精品） - 深入你需要的方向（后端、数据工程、DevOps） - 保持写 Pythonic 代码的习惯——能少写就少写

记住：Python 的优雅不是来自特性多，而是知道什么时候用什么。

```
import this
# The Zen of Python, by Tim Peters
# ...
# Simple is better than complex.
# ...
# Readability counts.
```